



INSTITUTO POLITÉCNICO DE BRAGANÇA Escola Superior de Tecnologia e de Gestão

Gestão de conflitos de preferências de utilizadores no ambiente

Veaceslav Popa - 47381

Miguel Viseu - 44803

Trabalho realizado sob a orientação de

Pedro Filipe Fernandes Oliveira

Paulo Matos

Licenciatura Engenharia Informática

2023-2024

Gestão de conflitos de preferências de utilizadores no ambiente

Relatório da UC de Projeto

Licenciatura em Engenharia Informática

Escola Superior de Tecnologia e Gestão

Veaceslav Popa - 47381

Miguel Viseu - 44803

A Escola de Tecnologia e de Gestão não se responsabiliza pelas opiniões expressas neste relatório.

Certifico que li este relatório e que, na minha opinião, é adequado no seu conteúdo e forma como demonstrador do trabalho desenvolvido no âmbito da UC de Projeto.

Pedro Filipe Fernandes Oliveira (Orientador)

Certifico que li este relatório e que, na minha opinião, é adequado no seu conteúdo e forma como demonstrador do trabalho desenvolvido no âmbito da UC de Projeto.

Paulo Matos (Coorientador)

Aceite para avaliação da UC de Projeto.

Dedicatória

Veaceslav

Dedico este trabalho à minha família que me apoiou no processo todo, especialmente aos meus pais e família e amigos.

Miguel

Dedico este projeto aos meus pais, irmão, sobrinhos e amigos, pelo apoio ao longo de todo o meu percurso. Sou grato por todo o suporte emocional, pelas palavras de encorajamento nos piores momentos e por acreditarem em mim incondicionalmente.

Agradecimentos

Gostaríamos de agradecer a todos aqueles que contribuíram para o sucesso deste projeto. Em especial, ao orientador deste trabalho, Professor Pedro Filipe Fernandes Oliveira, e ao coorientador, Professor Paulo Matos, pela disponibilidade, orientação, apoio e valiosas contribuições ao longo de todo o processo. O acompanhamento constante garantiu que o projeto seguisse um caminho coerente e alcançasse os objetivos estabelecidos.

Por fim, os nossos agradecimentos vão para todos aqueles que direta ou indiretamente contribuíram para o sucesso deste projeto.

Um sincero obrigado a todos!

Resumo

O presente relatório surge no âmbito da unidade curricular “Projeto” inserida na Licenciatura em Engenharia Informática do IPB, que decorreu sob a orientação do Professor Pedro Filipe Fernandes Oliveira e coorientação do Professor Paulo Matos. Este projeto resume os trabalhos, experiências e conhecimentos adquiridos durante estes 3 anos da licenciatura e tem como objetivo a construção de uma plataforma web que possibilite a gestão eficiente de conflitos de preferências de utilizadores em ambientes diversos, como casas, locais de trabalho e espaços públicos. A aplicação desenvolvida também terá funcionalidades adicionais, como a deteção e identificação automática das preferências dos utilizadores, a consideração de hierarquias de preferências e a integração de valores de segurança dos diferentes equipamentos atuadores. Isso permitirá uma administração abrangente e integrada das condições de conforto no ambiente, utilizando valências da inteligência artificial para tornar o sistema dinâmico e preditivo. Ao finalizar o desenvolvimento desta plataforma web, espera-se que a mesma proporcione uma solução robusta e eficiente para a gestão automática de preferências de conforto dos utilizadores, facilitando a adaptação das condições ambientais de forma personalizada e segura, sem necessidade de interação manual dos utilizadores.

Palavras-chave: Plataforma Web, Gestão de Conflitos, Preferências de Utilizadores, Conforto Ambiental, Inteligência Artificial.

Abstract

This report appears within the scope of the “Project” curricular unit included in the Degree in Computer Engineering at IPB, which took place under the guidance of Professor Pedro Filipe Fernandes Oliveira and co-supervision of Professor Paulo Matos. This project summarizes the work, experiences and knowledge acquired during these 3 years of the degree and aims to build a web platform that enables the efficient management of conflicts of user preferences in different environments, such as homes, workplaces and public spaces. The developed application will also have additional functionalities, such as the automatic detection and identification of user preferences, the consideration of preference hierarchies and the integration of safety values of different actuator equipment. This will allow comprehensive and integrated management of comfort conditions in the environment, using artificial intelligence capabilities to make the system dynamic and predictive. Upon completion of the development of this web platform, it is expected that it will provide a robust and efficient solution for the automatic management of users' comfort preferences, facilitating the adaptation of environmental conditions in a personalized and safe way, without the need for manual user interaction.

Keywords: Web Platform, Conflict Management, User Preferences, Environmental Comfort, Artificial Intelligence.

Conteúdo

1	Introdução.....	17
1.1	Enquadramento	17
1.2	Objetivos.....	17
1.3	Estrutura do Documento.....	18
2	Estado da Arte.....	19
2.1	Tecnologias/Ferramentas	19
2.1.1	Visual Studio Code.....	19
2.1.2	GitLab.....	20
2.1.3	ERDPlus	20
2.1.4	MySQL Server	20
2.1.5	MySQL WorkBench	21
2.1.6	Node.js	21
2.1.7	React.....	22
2.1.8	JavaScript.....	22
3	Análise de requisitos	23
3.1	Metodologias	23
3.2	Descrição do Problema e Proposta de Solução	23
3.2.1	Descrição do Problema	24
3.2.2	Proposta de Solução	24
3.3	Modelagem do Projeto	28
3.3.1	Diagrama de Casos de Uso	28
3.3.2	Fluxograma	28
3.3.3	Modelo Entidade Relacionamento.....	29
3.3.4	Mockups.....	31
4	Implementação.....	32
4.1	Implementação do código	32
4.1.1	Página Inicial	33
4.1.2	Página Ambiente	34
4.1.3	Cálculo da média	36
4.1.4	Página dos utilizadores	37

4.1.5	Página dos roles	38
4.1.6	Página dos registos	39
4.2	Dificuldade e soluções	40
4.3	Considerações finais	41
5	Testes e Usabilidade	42
5.1	Adicionar Ambientes.....	42
5.2	Adicionar Utilizadores.....	42
5.3	Adicionar Dispositivos a Ambientes	42
5.4	Limitar Acesso de Utilizadores a Ambientes	43
5.5	Listagens.....	43
5.6	Cálculo da fórmula	43
6	Conclusões.....	44
6.1	Resultados Alcançados	44
6.2	Lições Aprendidas	45
6.3	Melhorias Futuras	46
6.4	Conclusão	46
	Bibliografia.....	48
	Apêndice A- Instalação do MySQL Workbench e MySQL Community Server .	49
	Apêndice B- Instalação do Node.js	52
	Apêndice C- Configuração da conexão à base de dados.....	53
	Apêndice D– Configuração da API	55
	Apêndice E– Configuração dos Componentes	64
	Apêndice F- Configuração dos CRUDS	73
	Apêndice F- Configuração das Páginas	81
	Apêndice G- Proposta Original do Projeto.....	97
	Gestão de conflitos de preferências de utilizadores no ambiente.....	98

Lista de Figuras

Figura 1- Diagrama de Caso de Uso.....	28
Figura 2- Fluxograma	29
Figura 3- Modelo ER.....	30
Figura 4- Utilizadores do Sistema.....	33
Figura 5- Botão do Delete da página Environments	33
Figura 6- Página Environment	34
Figura 7- Página Adicionar Roles	35
Figura 8- Página Enviroment Apagar Ambiente.....	35
Figura 9- Média Ponderada.....	36
Figura 10- Página dos Utilizadores	37
Figura 11- Página Edit Utilizador	38
Figura 12- Página do Roles.....	38
Figura 13- Página Edit Role.....	39
Figura 14- Página dos Records.....	39
Figura 15- Criação Automática dos Registos.....	40
Figura 16- Instalação MySQL Workbench	49
Figura 17- Instalação MySQL	50
Figura 18- Instalação Node.js.....	52
Figura 19- Ligação com a base de dados	53
Figura 20- Múltiplas Rotas do servidor	54
Figura 21- API da Aplicação	55
Figura 22- API da página dos Ambientes.....	56
Figura 23- API da página das Preferências.....	57
Figura 24- API da página dos Dispositivos	58
Figura 25- API da página dos Checkins.....	59
Figura 26- API da página dos Registos	60
Figura 27- API da página dos Roles	61
Figura 28- API da página dos Roles Do Ambiente	62
Figura 29- API da página dos Utilizadores.....	63
Figura 30- Componente do AddDispositivoModal	64
Figura 31- Componente do EditDispositivoModal	65
Figura 32- Componente do AddUserModal.....	66
Figura 33- Componente do EditUserModal.....	67
Figura 34- Componente do AddUserModalCss	68
Figura 35- Componente do RoleForm	69
Figura 36- Componente do EditRoleModal.....	70
Figura 37- Componente do RoleCss.....	71
Figura 38- Componente do EditPreferenciasModal	72
Figura 39- Configuração do CRUD dos Ambientes.....	73
Figura 40- Configuração do CRUD dos Chekins	74
Figura 41- Configuração do CRUD dos Dispositivos	75

Figura 42- Configuração do CRUD das Preferências	76
Figura 43- Configuração do CRUD dos Registos	77
Figura 44- Configuração do CRUD dos Roles.....	78
Figura 45- Configuração do CRUD dos Roles do Ambiente.....	79
Figura 46- Configuração do CRUD dos Utilizadores	80
Figura 47- Configuração da página de EnterPageCss	81
Figura 48- Configuração da página dos Ambientes	82
Figura 49- Configuração da página do AmbientesList	83
Figura 50- Configuração da página do AmbientesListCss	84
Figura 51- Configuração da página do EnterAmbiente	85
Figura 52- Configuração da página dos Roles	86
Figura 53- Configuração da página do RolesList	87
Figura 54- Configuração da página do RolesCss	88
Figura 55- Configuração da página do UserList	89
Figura 56- Configuração da página do UserList	90
Figura 57- Configuração da página do UserListCss	91
Figura 58- Configuração da página do RegistosList	92
Figura 59- Configuração da página do RegistosCss	93
Figura 60- Configuração da página da App	94
Figura 61- Configuração da página do Backend da App	95
Figura 62- Configuração da página do AppCss.....	96

Lista de Tabelas

Tabela 1 – Tabela de testes 43

Siglas

API Application Programming Interface.

CSS Cascading Style Sheets.

ER Modelo Entidade Relacionamento.

ESTiG Escola Superior de Tecnologia e Gestão.

HTML HyperText Markup Language.

IDE Integrated Development Environment.

IoT Internet of Things.

IPB Instituto Politécnico de Bragança.

JSON JavaScript Object Notation.

SQL Structured Query Language.

URL Uniform Resource Locator.

Capítulo 1

1 Introdução

O projeto surgiu de uma necessidade que nos foi indicada pelo nosso orientador, onde era essencial criar uma solução que gerencie as preferências de conforto de diferentes utilizadores em um ambiente específico, como uma casa, local de trabalho ou espaço público. O principal objetivo é automatizar a adaptação das condições do ambiente às necessidades de cada utilizador. Nos capítulos seguintes iremos abordar ao pormenor de como implementamos este trabalho.

1.1 Enquadramento

O presente trabalho enquadra-se na unidade curricular de Projeto, relativo ao plano de estudo da Licenciatura de Engenharia Informática ministrado na Escola Superior de Tecnologia e Gestão (ESTiG) do Instituto Politécnico de Bragança (IPB). A proposta inicial do projeto encontra-se no apêndice A.

1.2 Objetivos

O principal objetivo deste trabalho é o desenvolvimento e implementação de uma solução, que dadas as preferências dos diferentes utilizadores num determinado ambiente/espço (casa, trabalho, espaço público), efetue a gestão de conflitos das preferências de cada utilizador, nomeadamente quanto às preferências de conforto (temperatura, humidade, luminosidade, musical, etc.) a aplicar num determinado espaço.

1.3 Estrutura do Documento

Este relatório está subdividido em capítulos, para que de uma forma organizada possa ser demonstrado todo o trabalho realizado:

- **Capítulo 1: Introdução**

Este capítulo faz uma abordagem de carácter introdutório com uma breve descrição do tema, o objetivo do projeto e a organização do documento.

- **Capítulo 2: Ferramentas e Tecnologias**

Neste capítulo é apresentado as ferramentas e tecnologias utilizadas para o desenvolvimento do projeto. As tecnologias e ferramentas representam o desenvolvimento de software de forma a ter um conhecimento das mesmas e a respetiva finalidade.

- **Capítulo 3: Análise de Requisitos**

Neste capítulo, foi discutida a abordagem para lidar com o problema, a análise detalhada desse problema e a criação de diagramas para termos uma visão mais detalhada e simplificada da plataforma.

- **Capítulo 4: Implementação**

Descreve-se detalhadamente o trabalho realizado, desde as configurações iniciais, as principais dificuldades e as próprias soluções para as resolver.

- **Capítulo 5: Testes/Avaliação/Discussão**

Aborda-se a fase de testes da aplicação e quais os métodos mais eficientes.

- **Capítulo 6: Conclusão**

No último capítulo, foram apresentadas as conclusões alcançadas com base nas análises e discussões anteriores, destacando os principais resultados, lições aprendidas e implementações futuras.

Capítulo 2

2 Estado da Arte

O presente capítulo baseia-se no estudo da literatura existente sobre gestão de conflitos de preferências de utilizadores em ambientes controlados, abordando temas como preferências ambientais, modelos de hierarquia de utilizadores, técnicas de análise de dados, integração com dispositivos IoT e algoritmos de otimização para resolução de conflitos. Foram analisadas diferentes fontes bibliográficas, que são referenciadas ao longo do capítulo e identificadas no final do documento.

2.1 Tecnologias/Ferramentas

Neste capítulo vão ser apresentadas as ferramentas e as tecnologias utilizadas durante o projeto.

2.1.1 Visual Studio Code

O Visual Studio Code é um ambiente de desenvolvimento integrado, Integrated Development Environment (IDE), amplamente utilizado no desenvolvimento de software. É uma ferramenta poderosa e versátil, com recursos que facilitam a escrita de código, a depuração e a gestão de projetos. Suporta várias linguagens de programação, incluindo HyperText Markup Language (HTML), Cascading Style Sheets (CSS), JavaScript e outras tecnologias web utilizadas no desenvolvimento da plataforma. Uma das principais vantagens do Visual Studio Code é a extensibilidade. Possui uma vasta biblioteca de extensões disponíveis, desenvolvidas pela comunidade, que adicionam recursos e funcionalidades extras ao IDE.

Além disso, o Visual Studio Code possui integração com sistemas de controle de versão, como Git, facilitando a gestão e o rastreamento das alterações de código ao longo do desenvolvimento do projeto. Essa integração permitiu trabalhar de forma mais

colaborativa e acompanhar as alterações feitas. [1]

2.1.2 GitLab

No decorrer do projeto, o GitLab foi utilizado como uma plataforma de hospedagem de código-fonte e manutenção de repositórios. O GitLab oferece um conjunto de ferramentas colaborativas que facilitam o trabalho em equipa.

Uma das principais vantagens do GitLab é a possibilidade de hospedar repositórios Git de forma privada. Isso significa que podemos compartilhar o código-fonte do projeto apenas com as pessoas autorizadas, garantindo a privacidade e a segurança do código.

No contexto do projeto de desenvolvimento da plataforma de gestão de equipamentos, o GitLab foi utilizado como a plataforma para hospedar o repositório Git contendo o código-fonte do projeto. Isto facilitou a colaboração, permitindo o controle de versão eficiente e contribuiu para a entrega contínua do projeto. [2]

2.1.3 ERDPlus

É uma ferramenta online gratuita que serviu para criar o diagrama de base de dados. Ele é usado para visualizar a estrutura dos dados e os relacionamentos entre eles, definir chaves primárias e estrangeiras, e facilitar a normalização das tabelas. Além de transformar diagramas ER em esquemas relacionais, o ERDPlus permite a modelagem de armazém de dados através de diagramas de estrela, que incluem tabelas de fatos e dimensões. A ferramenta é fácil de usar, com uma interface intuitiva baseada em arrastar e soltar, e não requer instalação, sendo acessível diretamente pelo navegador. Ela também oferece opções para salvar e exportar diagramas em vários formatos, como PDF e SQL, e facilita a colaboração e o compartilhamento de diagramas entre os utilizadores. [3]

2.1.4 MySQL Server

MySQL Server é o principal componente do sistema de gestão de base de dados MySQL, encarregado de armazenar, gerir e fornecer acesso aos dados com integridade e consistência, mesmo com múltiplos utilizadores e aplicações a aceder simultaneamente. Utilizando SQL, permite operações de inserção, atualização, exclusão e recuperação de dados. Suporta transações ACID para garantir operações seguras, emprega controlo de concorrência e bloqueio, e oferece robusta segurança com autenticação e criptografia.

Para alta disponibilidade, possui replicação de dados e ferramentas de backup. Escalável e compatível com diversas plataformas, é amplamente usado em aplicações web, sistemas empresariais e data warehouses, oferecendo desempenho, segurança e flexibilidade. [4]

2.1.5 MySQL WorkBench

O MySQL Workbench é uma ferramenta visual desenvolvida pela Oracle Corporation para o design e administração de bases de dados MySQL. Com uma interface gráfica unificada, facilita a criação, administração e manutenção de bases de dados. Permite a modelação visual através de diagramas EER, oferece um editor de SQL avançado com destaque de sintaxe e autocompletar, e facilita a execução de consultas, visualização de resultados e exportação de dados. A ferramenta também simplifica a administração de bases de dados, gerindo contas de utilizador e privilégios, e disponibiliza ferramentas para backup, recuperação de dados e monitorização do desempenho do servidor. Suporta a migração de esquemas e dados de outros sistemas de gestão de bases de dados para MySQL e permite a navegação e visualização estruturada dos dados, além da criação de relatórios e análises. [5]

2.1.6 Node.js

Node.js é um ambiente de execução de código JavaScript do lado do servidor. Permite aos programadores utilizar JavaScript não apenas para aplicações web no navegador, mas também para criar e gerir serviços e aplicações do lado do servidor. A sua arquitetura assíncrona e baseada em eventos facilita o tratamento eficiente de operações de entrada e saída, sendo ideal para aplicações que necessitam lidar com múltiplas conexões simultâneas. Node.js é amplamente reconhecido pela sua performance e escalabilidade, sendo utilizado em aplicações em tempo real, APIs e microserviços com um vasto ecossistema de módulos disponíveis via NPM, simplifica o desenvolvimento e a integração de funcionalidades adicionais, permitindo o uso unificado de JavaScript em todo o stack de uma aplicação, Node.js simplifica a colaboração e o compartilhamento de código entre diferentes partes do sistema, expandindo significativamente as capacidades de desenvolvimento backend com JavaScript. [6]

2.1.7 React

O React foi utilizado como framework web principal. React é uma biblioteca JavaScript criada para construir interfaces de utilizador interativas e dinâmicas em web e dispositivos móveis. Ideal para aplicações de página única (SPA), React carrega conteúdo dinamicamente, proporcionando uma experiência fluida. A sua principal vantagem é a criação de componentes modulares e reutilizáveis, facilitando a manutenção e a escalabilidade do código. Ferramentas como Redux e a Context API ajudam na gestão eficiente do estado, enquanto hooks como useState e useEffect simplificam a lógica dos componentes. React também suporta renderização no lado do servidor (SSR) com frameworks como Next.js, melhorando a performance e o SEO. É amplamente utilizado em dashboards, e-commerce, redes sociais, aplicações de mensagens e formulários complexos, devido à sua capacidade de atualizar a interface eficientemente. [7]

2.1.8 JavaScript

O JavaScript foi utilizado para adicionar interatividade e funcionalidade dinâmica às páginas web. O JavaScript é uma linguagem de programação de alto nível, amplamente usada no desenvolvimento web, que permite manipular elementos da página, responder a eventos e interagir com o utilizador, tornando a plataforma mais dinâmica e responsiva. O JavaScript foi utilizado para manipular elementos da página em tempo real. Por exemplo, ao selecionar uma opção no menu suspenso, o JavaScript foi responsável por mostrar ou ocultar determinados elementos ou atualizar o conteúdo dinamicamente, sem a necessidade de recarregar a página. Isso tornou a navegação mais suave e rápida para os utilizadores. [8]

Capítulo 3

3 Análise de requisitos

Neste capítulo, será apresentada a abordagem geral adotada para o desenvolvimento do projeto. Serão discutidos os principais aspectos abordados durante o processo, desde a análise do problema até à implementação da solução.

3.1 Metodologias

No desenvolvimento de qualquer aplicação é importante adotar uma metodologia. Estes métodos podem ser vistos como um processo ou etapas necessárias a cumprir desde o início do desenvolvimento da aplicação até à sua utilização. O presente projeto seguiu uma metodologia composta pelas seguintes fases:

- Organização da informação e objetivos da aplicação.
- Seleção de Linguagem de programação e possíveis frameworks a utilizar.
- Desenvolvimento da aplicação.
- Implementação e testes, em contexto real.
- Possíveis melhorias, após testes.
- Elaboração do relatório.

3.2 Descrição do Problema e Proposta de Solução

Esta alínea tem como objetivo apresentar uma descrição detalhada do problema enfrentado pela organização em relação à gestão de conflitos de preferências de utilizadores no ambiente. Além disso, será discutida a proposta de solução por meio

da implementação de uma plataforma web, destacando os principais recursos e benefícios.

3.2.1 Descrição do Problema

A mobilidade diária no cotidiano do ser humano, aliada à atratividade do aumento da automatização das tarefas quotidianas, vem também no contexto das casas inteligentes (Smart Homes) criar a necessidade da detecção e identificação das preferências do utilizador de forma automatizada para assim poder ser realizada a consequente adaptação das condições de conforto do ambiente/espço aos utilizadores presentes. Neste contexto e para ir de encontro à automatização de todo o processo, pretende-se o desenvolvimento de uma solução que permita de forma automática e sem qualquer interação do utilizador, a gestão de conflitos das diferentes preferências de cada utilizador que se encontre no espaço, para tal devem ser tidas em conta as diferentes hierarquias, assim como valores de segurança dos diferentes equipamentos atuadores (ar condicionado, radiadores, ventiladores, etc.) presentes no espaço. Para o desenvolvimento pretende-se tirar partido das diferentes valências da inteligência artificial, para tornar a solução dinâmica e preditiva.

3.2.2 Proposta de Solução

A proposta de solução para os problemas mencionados são os seguintes:

1. Introdução

Desenvolvimento usando React para a interface do usuário, Node.js para o backend e API em JavaScript, juntamente com uma base de dados MySQL.

2. Estrutura da Base de Dados

A base de dados foi projetada para suportar a hierarquia de usuários e suas preferências. Abaixo está uma descrição mais coerente da estrutura e o propósito de cada tabela.

Tabela ambientes:

- Armazena informações sobre diferentes ambientes monitorados.
- Inclui detalhes como tipo de ambiente, preferências de temperatura, brilho, umidade e som.

Tabela roles:

- Define os diferentes papéis dos usuários e os pesos associados a cada papel, que influenciam a ponderação das preferências.

Tabela utilizadores:

- Armazena informações dos usuários, incluindo o papel associado.
- Inclui um indicador se o usuário está presente no ambiente.

Tabela checkins:

- Regista a entrada e saída dos usuários dos ambientes.
- Facilita o rastreamento de quais usuários estão presentes em um dado momento.

Tabela dispositivos:

- Inclui o estado e medições como temperatura e decibéis.

Tabela eventos:

- Regista eventos relacionados aos dispositivos, ajudando no monitoramento e na manutenção.

Tabela preferências:

- Armazena as preferências individuais dos usuários para temperatura, brilho, humidade e som.

Tabela registros:

- Mantém um histórico das temperaturas calculadas para os ambientes com base nas preferências ponderadas dos usuários presentes.

Tabela roles_do_ambiente:

- Relaciona os papéis aos ambientes específicos, permitindo configuração diferenciada conforme o ambiente.

3. Desenvolvimento da Solução**Frontend (React)**

- Criação de uma interface de usuário intuitiva para gerenciamento de ambientes, usuários e dispositivos.
- Implementação de dashboards para visualização em tempo real dos estados dos dispositivos e das condições dos ambientes.
- Formulários para a entrada e edição de dados, como preferências de usuários e detalhes de dispositivos.

Backend (Node.js)

- Desenvolvimento de APIs RESTful para comunicação entre o frontend e a base de dados.
- Implementação de lógica de negócios para cálculo das médias ponderadas das preferências de temperatura.

- Gerenciamento de autenticação e autorização dos usuários com base em seus papéis.
- Manipulação de eventos dos dispositivos IoT e atualização dos estados na base de dados.

Integração com IoT

- Configuração de dispositivos IoT para enviar dados periodicamente ao backend.
- Implementação de serviços de escuta no Node.js para receber e processar dados dos dispositivos.
- Atualização automática das condições dos ambientes com base nos dados recebidos.

4. Lógica de Cálculo da Temperatura Ponderada

Para calcular a temperatura preferida de um ambiente com base nas preferências dos usuários presentes, foi utilizada a seguinte fórmula:

Média Ponderada

$$\frac{\sum(\text{preferencia do usuario} * \text{peso do role}) + (\text{preferencia do ambiente} * \text{peso da preferencia})}{\sum(\text{peso do role}) + \text{peso da preferencia}}$$

5. Conclusão

A solução implementada oferece um sistema de automação robusto e flexível, permitindo o controle eficiente de ambientes com base nas preferências dos usuários. Utilizando uma combinação de React, Node.js e MySQL, conseguimos integrar dispositivos IoT de forma eficaz, garantindo um ambiente confortável e ajustado conforme as necessidades dos usuários presentes. Esta arquitetura modular e escalável permite futuras expansões e personalizações conforme as necessidades específicas dos usuários e do ambiente evoluam.

3.3 Modelagem do Projeto

A modelagem do projeto por meio de diagramas de casos de uso, Modelo Entidade Relacionamento (ER) e mockups proporcionou uma visão detalhada e estruturada da plataforma de gestão de equipamentos. Essas representações ajudaram a definir as funcionalidades, as entidades, os relacionamentos e a aparência da interface, auxiliando na organização e no desenvolvimento eficiente do sistema.

3.3.1 Diagrama de Casos de Uso

Para a modelagem do projeto da plataforma de gestão de preferências em ambientes inteligentes foram utilizados os diagramas de casos de uso. Esses diagramas são uma representação visual das interações entre os atores (utilizadores ou sistemas externos) e o sistema em questão.

O diagrama de casos de uso é apresentado na Figura 1.

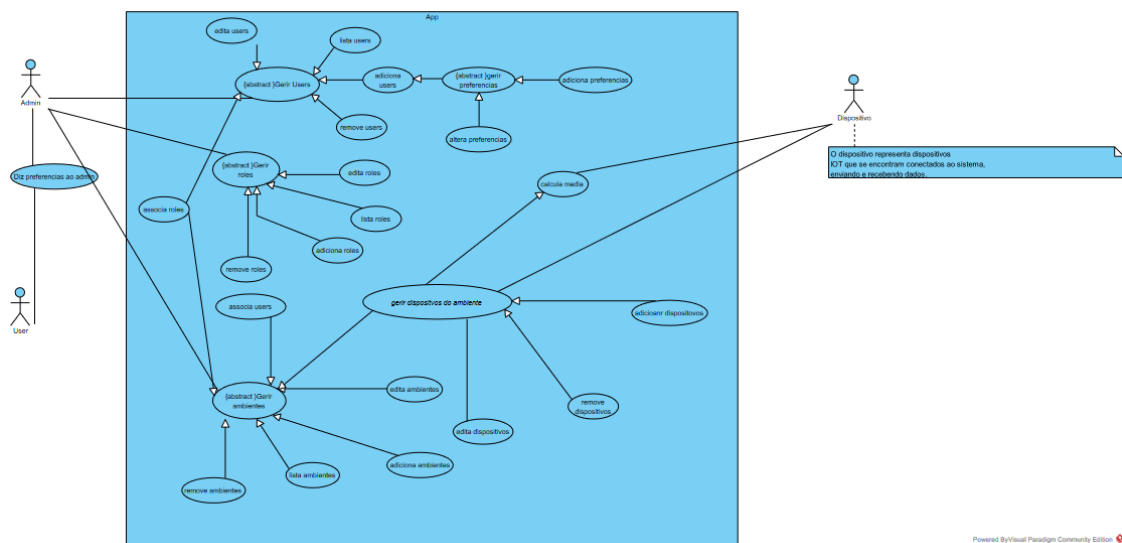


Figura 1- Diagrama de Caso de Uso

3.3.2 Fluxograma

Para esta etapa, foi criado um fluxograma onde detalha um processo robusto para garantir que um ambiente está corretamente configurado com todos os dispositivos

necessários e que os utilizadores têm os papéis apropriados. Ele também garante que não haja conflitos em outros ambientes, promovendo uma gestão eficiente e segura do ambiente de trabalho. Ver a figura 3.

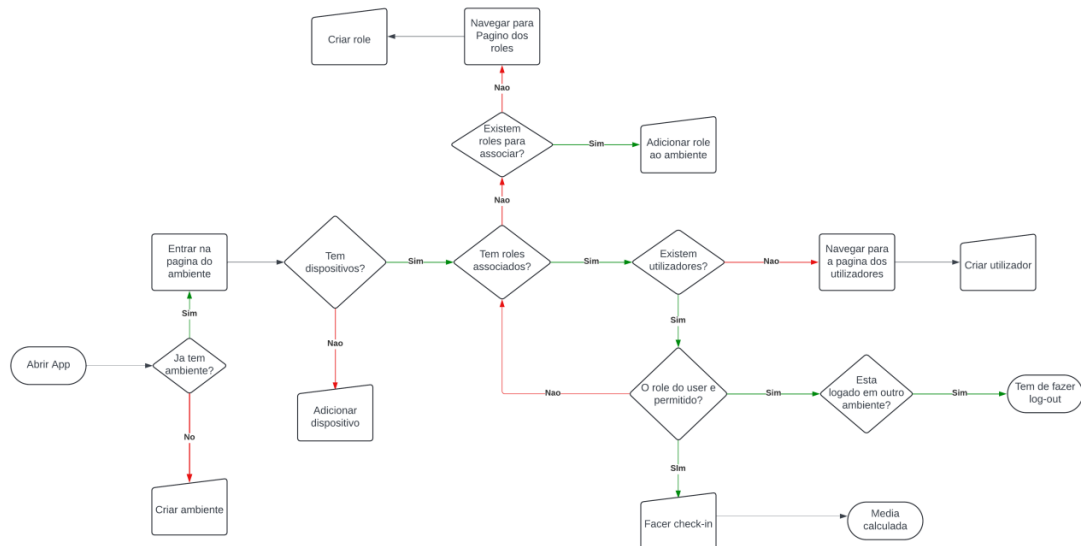


Figura 2- Fluxograma

3.3.3 Modelo Entidade Relacionamento

Outra etapa importante da modelagem do projeto foi a criação do modelo ER, usado para representar as entidades do sistema, seus atributos e relacionamentos. As principais entidades identificadas foram Ambientes, Usuários, Dispositivos, Roles, Preferências e Registos, com atributos como Nome_Usuario, Temperatura e Calculated_temperature, entre outros.

Os relacionamentos foram estabelecidos da seguinte forma: roles_do_ambiente relaciona Roles e Ambientes, checkins relaciona Usuários e Ambientes, registros relaciona Usuários e Ambientes, eventos relaciona Dispositivos com Eventos específicos, preferências relaciona Usuários com suas preferências, dispositivos relaciona Ambientes com Dispositivos, e utilizadores relaciona Roles com Usuários. Esses relacionamentos são representados por meio de chaves estrangeiras no modelo ER. Ver a figura 2.

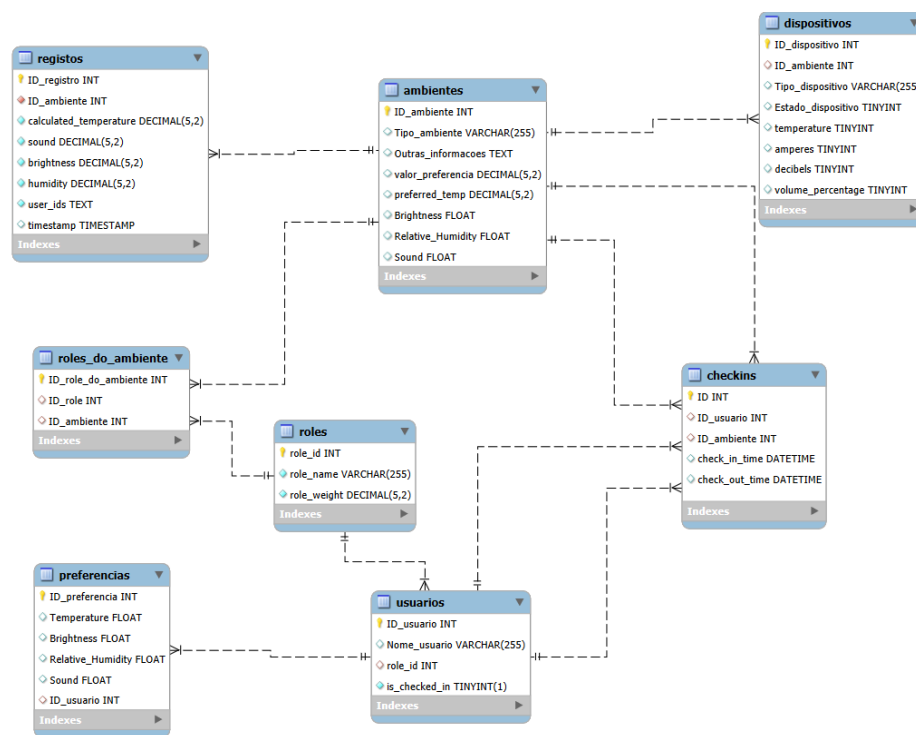


Figura 3- Modelo ER

O modelo ER forneceu uma estrutura sólida para a organização e o armazenamento dos dados no sistema. Eles serviram como base para a criação de uma base de dados, auxiliando no desenvolvimento das tabelas e relacionamentos necessários para armazenar as informações de forma eficiente e coerente.

3.3.4 Mockups

Os mockups foram criados para representar visualmente as páginas e a estrutura da plataforma. Permitiram uma visualização prévia do design e da disposição dos elementos no ecrã, garantindo consistência e fidelidade ao design proposto. Foram criados mockups para as principais páginas da plataforma, como a página inicial (em que irá ser a nossa página do Environments List) irá conter também a página do Users List, Roles List e Records List. Cada mockup apresenta os elementos da interface, como botões, campos de entrada, tabelas e menus, de forma a representar a experiência do utilizador durante a interação com o sistema.

Capítulo 4

4 Implementação

Neste capítulo, será abordado o processo de desenvolvimento e implementação do sistema. Serão destacados os pontos mais relevantes da implementação, incluindo as dificuldades encontradas e as soluções desenvolvidas ou aplicadas. Será feita uma distinção clara entre as funcionalidades originais e o que foi necessário implementar para atender às necessidades específicas do projeto. [9]

4.1 Implementação do código

Durante a implementação, foram utilizadas várias linguagens de programação como já referidas anteriormente no capítulo 2. O Node.js foi o framework escolhido para o desenvolvimento do backend do sistema, devido à sua robustez e facilidade de uso. O código foi estruturado em módulos e classes, seguindo as melhores práticas de programação. [10]

4.1.1 Página Inicial

A página inicial, também conhecida como home page, é uma parte crucial do sistema, pois fornece aos utilizadores uma visão geral dos ambientes:

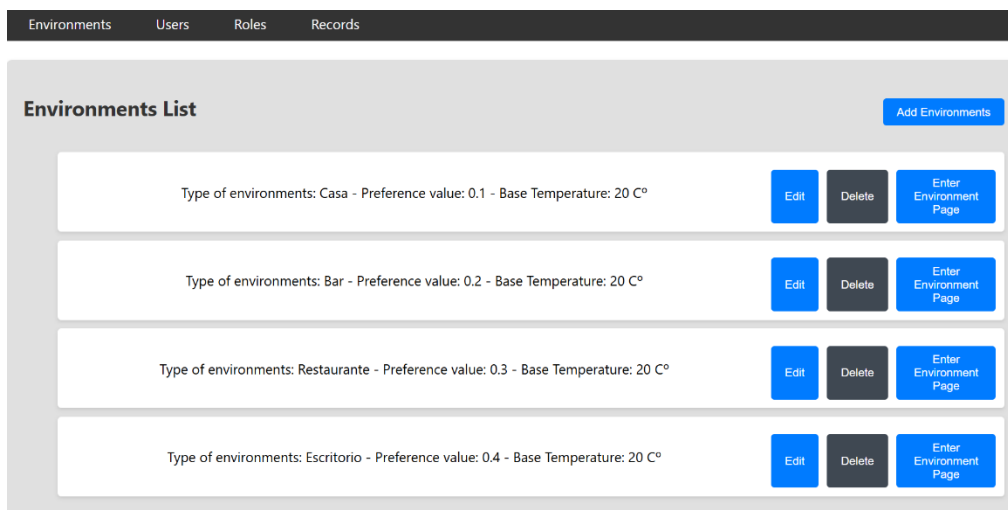


Figura 4- Utilizadores do Sistema

Composta por um menu que permite navegar para as diferentes páginas do sistema e apresenta um componente de cabeçalho, este componente mostra a opção de ir para a página dos utilizadores (users), roles e registos. Para além disso apresenta a opção de adicionar um novo ambiente definindo as preferências para cada ambiente e o peso da preferência. Também permite editar um ambiente que já existe. Também tem a opção de apagar o ambiente caso confirme o pop-up:

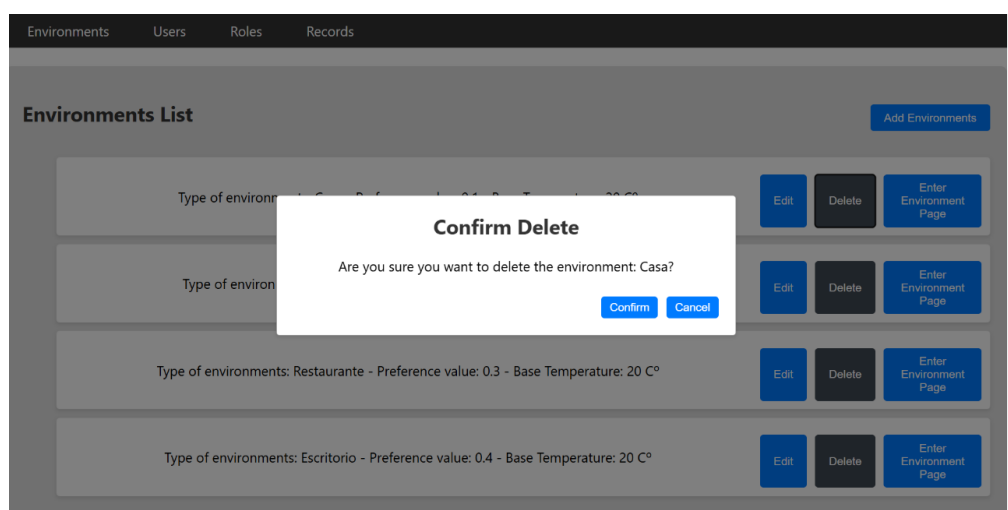


Figura 5- Botão do Delete da página Environments

A opção “Enter Enviroment Page” permite ao utilizador aceder a página do ambiente(enviroment).

4.1.2 Página Ambiente

Esta página permite ao utilizador adicionar roles a um ambiente de forma dinâmica a que só se o role esta associado ao ambiente é possível fazer check-in a um utilizador, caso não este com check-in em outro ambiente. Apos o utilizador ter feito o check-in no ambiente as suas preferências entram na fórmula para calcular através de uma média ponderada a temperatura, luminosidade, humidade e volume de um certo aparelho.

Environments Users Roles Records

Environment Page

Add Role to Environment

Select a role

Allowed Users

Select a user

Checked In Users

- Alberto Esteves - regular - 0.1 -
- Maria Albera - Chef - 0.4 -
- Carlos Jose - admin - 0.45 -

Devices

Calculated Temp: [21.6]

Figura 6- Página Environment

Os aparelhos apresentam 5 variáveis: temperature, humidade, luminosidade(ampers), volume e o estado do dispositivo (state). Dependendo de que variáveis o dispositivo tem controlo sobre e feita a media pondera para essa variável. Se o dispositivo estiver desligado não é feita a media.

The image shows a web application interface with a modal window titled "Add Device". The modal contains several input fields and dropdown menus. At the top, there is a "Type:" label followed by an empty text input field. Below this are five control settings, each with a label and a dropdown menu: "Temperature Control:" (set to "No Control"), "Amperes Control:" (set to "No Control"), "Decibels Control:" (set to "No Control"), "Volume Control:" (set to "No Control"), and "State (ON/OFF):" (set to "OFF"). At the bottom of the modal are two blue buttons: "Add Device" and "Cancel". The background of the application is dimmed, showing a list of users with names like "Maria Albera" and "Carlos Jose" and a "Check Out" button.

• Maria Albera - Chef - 0.4 - [Check Out](#)

• Carlos Jose - admin - 0.45 - [Check Out](#)

Add Device

Type:

Temperature Control: No Control ▾

Amperes Control: No Control ▾

Decibels Control: No Control ▾

Volume Control: No Control ▾

State (ON/OFF): OFF ▾

[Add Device](#) [Cancel](#)

• regular [Delete](#)

Figura 7- Página Adicionar Roles

É possível também eliminar os dispositivos de um ambiente e editá-los:

The image shows a web application interface for managing devices and roles. At the top, there is a list of users with names like "Maria Albera" and "Carlos Jose" and a "Check Out" button. Below this is a section titled "Devices" which contains a table of device data. The table has columns for "AC Calculated Temp", "Calculated Brightness %", "Calculated Humidity %", and "Calculated Sound %". The data row shows values: [21.6], [], [], and []. Below the table are two blue buttons: "Delete" and "Edit". Below the "Devices" section is a blue button labeled "Add Device". Below this is a section titled "Roles" which contains a table of roles. The table has columns for "Role" and "Delete". The data row shows the role "admin" and a "Delete" button. Below the "Roles" section is a blue button labeled "Delete".

• Maria Albera - Chef - 0.4 - [Check Out](#)

• Carlos Jose - admin - 0.45 - [Check Out](#)

Devices

AC Calculated Temp: [21.6]
Calculated Brightness % : []
Calculated Humidity % : []
Calculated Sound % : []

[Delete](#) [Edit](#)

[Add Device](#)

Roles

admin	Delete
regular	Delete

Figura 8- Página Enviroment Apagar Ambiente

4.1.3 Cálculo da média

O cálculo da média foi feito através de uma média ponderada prévia. Foi utilizado uma query select a base de dados:

```
70
71 // Device temp by ambiente id
72 router.get('/:id', (req, res) => {
73   const { id } = req.params;
74   const query = `
75   SELECT
76     d.ID_dispositivo,
77     d.Tipo_dispositivo,
78     d.temperature,
79     d.amperes,
80     d.decibels,
81     d.volume_percentage,
82     a.preferred_temp,
83     a.valor_preferencia,
84     a.Brightness,
85     a.Relative_humidity,
86     a.Sound,
87   ROUND(
88     SUM(CASE WHEN d.temperature = 1 THEN (p.Temperature * r.role_weight) + (a.preferred_temp * a.valor_preferencia) ELSE 0 END) /
89     SUM(CASE WHEN d.temperature = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_temperature,
90   ROUND(
91     SUM(CASE WHEN d.amperes = 1 THEN (p.Brightness * r.role_weight) + (a.Brightness * a.valor_preferencia) ELSE 0 END) /
92     SUM(CASE WHEN d.amperes = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_brightness,
93   ROUND(
94     SUM(CASE WHEN d.decibels = 1 THEN (p.Relative_Humidity * r.role_weight) + (a.Relative_Humidity * a.valor_preferencia) ELSE 0 END) /
95     SUM(CASE WHEN d.decibels = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_relative_humidity,
96   ROUND(
97     SUM(CASE WHEN d.volume_percentage = 1 THEN (p.Sound * r.role_weight) + (a.Sound * a.valor_preferencia) ELSE 0 END) /
98     SUM(CASE WHEN d.volume_percentage = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_sound
99   FROM
100     dispositivos d
101   JOIN
102     ambientes a ON d.ID_ambiente = a.ID_ambiente
103   JOIN
104     checkins c ON a.ID_ambiente = c.ID_ambiente
105   JOIN
106     preferencias p ON c.ID_usuario = p.ID_usuario
107   JOIN
108     usuarios u ON p.ID_usuario = u.ID_usuario
109   JOIN
110     roles r ON u.role_id = r.role_id
111   WHERE
112     c.check_out_time IS NULL AND a.ID_ambiente = ?
113   GROUP BY
114     d.ID_dispositivo;
115   `;
116
117   console.log('Executing query with ID_ambiente:', id);
118
119   pool.query(query, [id], (error, results) => {
120     if (error) {
121       console.error('Error fetching devices by ambiente ID:', error);
122       res.status(500).send('Internal Server Error');
123       return;
124     }
125
126     // Log the results to the console
127     console.log('Calculated values:', results);
128     res.status(200).send(results);
129   });
130 });
131
132
```

Figura 9- Média Ponderada

É feito um select através de joins entre tabelas onde para cada ambiente é verificado os utilizadores que estão com check-in no ambiente através de uma união com a tabela checkins, e para cada utilizador e feito outra união com a tabela preferências onde estão guardadas as preferências de cada utilizador, faz também uma união com a tabela roles para seleccionar o peso do role. Apos a agrupar esses dados a equação faz um somatório das preferências de cada utilizador multiplicado pelo peso do role que lhe foi atribuído mais a preferência do ambiente multiplicado pelo peso da sua preferência, a dividir

somatório peso do role mais o peso da preferência do ambiente. É também verificado se o dispositivo tem controlo sobre essa variável.

4.1.4 Página dos utilizadores

Esta página é responsável por exibir todos os utilizadores que foram criados na app, permite também adicionar um utilizador, editar ou apagar(delete).



User List		Add User
Carlos Jose - admin- Role Weigth: 0.45	Edit	Delete
Manel Carlos - funcionario- Role Weigth: 0.2	Edit	Delete
Alberto Esteves - regular- Role Weigth: 0.1	Edit	Delete
Maria Albera - Chef - Role Weigth: 0.4	Edit	Delete
Marco Ribeiro - funcionario- Role Weigth: 0.2	Edit	Delete
Maria Esteves - funcionario- Role Weigth: 0.2	Edit	Delete

Figura 10- Página dos Utilizadores

Foram feitas validações de formulário para evitar que sejam introduzidos valores que ponham em causa o funcionamento devido da fórmula. O formulário pop-up não permite que sejam introduzidos valores na temperatura maiores que 30 ou menores que 15, preferências de brilho, som ou humidade só são aceites valores numéricos entre 0 e 100. Também é obrigatório associar um role ao utilizador.

Edit User

User Name: Carlos Jose

Role: Select a role !
Role is required

Preferred Temperature: 120 !
Preferred Temperature must be at most 30

Preferred Brightness: 150 !
Preferred Brightness must be at most 100

Preferred Relative Humidity: 20

Preferred Sound:

Figura 11- Página Edit Utilizador

4.1.5 Página dos roles

Semelhante a página dos utilizadores a página dos roles permite editar, adicionar ou remover roles consoante a necessidade.

Roles List		
admin - 0.45	Edit	Remove
funcionario - 0.2	Edit	Remove
regular - 0.1	Edit	Remove
Chef - 0.4	Edit	Remove
Nao Regular - 0.1	Edit	Remove

Figura 12- Página do Roles

O formulário dos roles também é um pop-up, com uma validação que impede que o valor da preferência seja maior que 1 e menor que 0 para evitar que a fórmula não funcione corretamente.

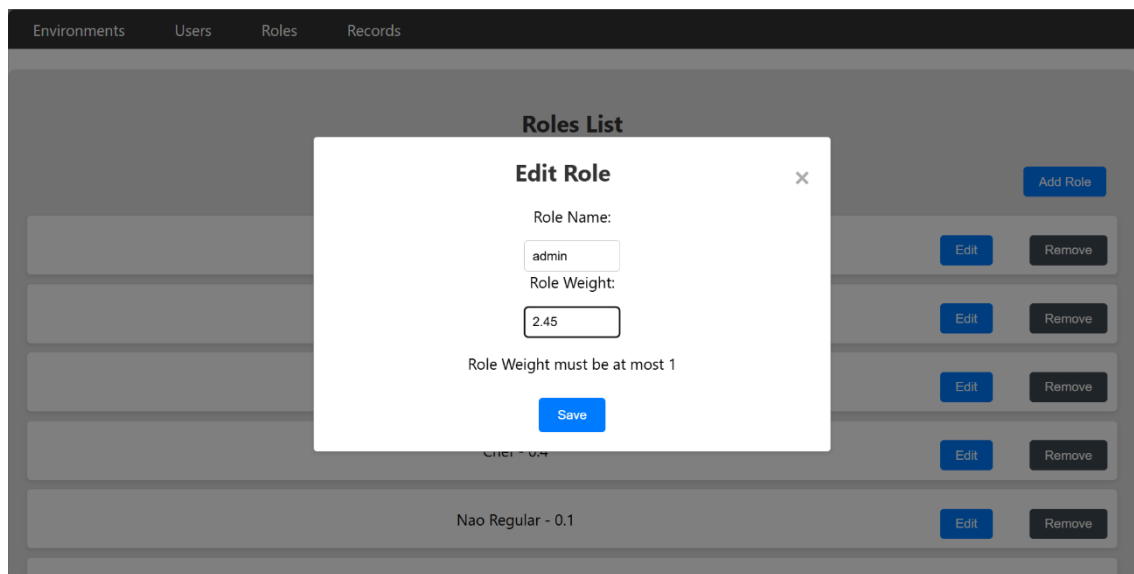


Figura 13- Página Edit Role

4.1.6 Página dos registos

A página dos registos permite visualizar as medias ponderadas para cada variável de um dispositivo, para cada ambiente e os utilizadores que fizeram checkin.

Environments		Users	Roles	Records
Registros List				
Ambiente ID: 2 - Calculated Temperature: 21.6 C° - Sound: 30.8 - Brightness: 38 - Humidity: 20- Date: 2024-06-24T14:35:51.000Z				Delete
Ambiente ID: 2 - Calculated Temperature: 21.6 C° - Sound: 30.8 - Brightness: 38 - Humidity: 20- Date: 2024-06-24T16:30:45.000Z				Delete
Ambiente ID: 2 - Calculated Temperature: 21.6 C° - Sound: 30.8 - Brightness: 38 - Humidity: 20- Date: 2024-06-24T16:35:51.000Z				Delete
Ambiente ID: 2 - Calculated Temperature: 21.6 C° - Sound: 30.8 - Brightness: 38 - Humidity: 20- Date: 2024-06-24T17:05:51.000Z				Delete
Ambiente ID: 2 - Calculated Temperature: 21.6 C° - Sound: 30.8 - Brightness: 38 - Humidity: 20- Date: 2024-06-24T19:22:18.000Z				Delete
Ambiente ID: 2 - Calculated Temperature: 21.6 C° - Sound: 30.8 - Brightness: 38 - Humidity: 20- Date: 2024-06-24T19:35:51.000Z				Delete

Figura 14- Página dos Records

Permite apenas apagar(delete) um registo pois são gerados automaticamente pela base de dados, e a edição é redundante. Foi usado o seguinte código para implementar a criação automática dos registos:

```
1 SET GLOBAL event_scheduler = ON;
2 SHOW VARIABLES LIKE 'event_scheduler';
3 use mydb;
4 CREATE EVENT UpdateRegistosEvent
5 ON SCHEDULE EVERY 30 MINUTE
6 DO
7     CALL UpdateRegistos();
8 DELIMITER //
9 CREATE PROCEDURE UpdateRegistos()
10 BEGIN
11     DECLARE done INT DEFAULT FALSE;
12     DECLARE ambiente_id INT;
13     DECLARE ambiente_cursor CURSOR FOR SELECT ID_ambiente FROM ambientes;
14     DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
15     OPEN ambiente_cursor;
16     read_loop: LOOP
17         FETCH ambiente_cursor INTO ambiente_id;
18         IF done THEN
19             LEAVE read_loop;
20         END IF;
21         INSERT INTO registros (ID_ambiente, calculated_temperature, user_ids, timestamp, sound, brightness, humidity)
22         SELECT
23             ambiente_id,
24             ROUND(
25                 SUM(CASE WHEN d.temperature = 1 THEN (p.Temperature * r.role_weight) + (a.preferred_temp * a.valor_preferencia) ELSE 0 END) /
26                 SUM(CASE WHEN d.temperature = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_temperature,
27             GROUP_CONCAT(DISTINCT c.ID_usuario) AS user_ids,
28             CURRENT_TIMESTAMP,
29             ROUND(
30                 SUM(CASE WHEN d.volume_percentage = 1 THEN (p.Sound * r.role_weight) + (a.Sound * a.valor_preferencia) ELSE 0 END) /
31                 SUM(CASE WHEN d.volume_percentage = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_sound,
32             ROUND(
33                 SUM(CASE WHEN d.amperes = 1 THEN (p.Brightness * r.role_weight) + (a.Brightness * a.valor_preferencia) ELSE 0 END) /
34                 SUM(CASE WHEN d.amperes = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_brightness,
35             ROUND(
36                 SUM(CASE WHEN d.decibels = 1 THEN (p.Relative_Humidity * r.role_weight) + (a.Relative_Humidity * a.valor_preferencia) ELSE 0 END) /
37                 SUM(CASE WHEN d.decibels = 1 THEN r.role_weight + a.valor_preferencia ELSE 0 END), 1) AS calculated_relative_humidity
38         FROM
39             dispositivos d
40         JOIN
41             ambientes a ON d.ID_ambiente = a.ID_ambiente
42         JOIN
43             checkins c ON a.ID_ambiente = c.ID_ambiente
44         JOIN
45             preferencias p ON c.ID_usuario = p.ID_usuario
46         JOIN
47             usuarios u ON p.ID_usuario = u.ID_usuario
48         JOIN
49             roles r ON u.role_id = r.role_id
50         WHERE
51             c.check_out_time IS NULL AND a.ID_ambiente = ambiente_id
52         GROUP BY
53             d.ID_dispositivo;
54     END LOOP;
55     CLOSE ambiente_cursor;
56 END //
57 DELIMITER ;
```

Figura 15- Criação Automática dos Registos

4.2 Dificuldade e soluções

Grande parte do problema foi criar uma base de dados relacional em que seriam feitos pedidos de queries sql através de uma API. A implementação da solução foi feita com a ideia de escalabilidade em mente. Outra dificuldade que tivemos foi implementar uma

tabela onde seria apenas ponderado para a media utilizadores que estivessem no mesmo espaço por mais de 15 minutos.

4.3 Considerações finais

Neste capítulo, foi abordado o processo de desenvolvimento e implementação do sistema, destacando os pontos mais relevantes, as dificuldades encontradas e as soluções técnicas inovadoras aplicadas. Foi dada uma distinção clara entre as funcionalidades originais do código de terceiros e as implementações personalizadas necessárias para

Capítulo 5

5 Testes e Usabilidade

Este capítulo apresenta os testes que nós fomos fazendo ao longo do tempo, tanto para resolução de erros de lógica tanto para melhorias pequenas ao aspeto visual.

5.1 Adicionar Ambientes

Os testes que fizemos para adicionar Ambientes foram relativamente simples, pois um ambiente tem apenas um tipo, que foi implementado primeiramente em tipo texto livre, que após ponderação foi alterado para uma dropdown que tem um número limitado de tipo de ambientes possíveis de seleccionar. Uma preferência de 0 a 1 que inicialmente não existia, mas foi implementa juntamente com as preferências do ambiente para tornar mais dinâmica a solução do problema. Como o ambiente e a tabela que abrange todas as outras foi relativamente simples de implementar.

5.2 Adicionar Utilizadores

Inicialmente cada utilizador estava associado a um ambiente quando era criado, mas apos testar essa abordagem chegamos a conclusão que limitava a nossa solução e mudamos para uma abordagem diferente em que consistia em usar uma tabela intermedia chama checkins para associar utilizadores a um ambiente. Inicialmente estava diretamente associado aos utilizadores as suas preferências o que tornava a tabela algo grande, por isso mudamos as preferências do utilizador para uma tabela preferências.

5.3 Adicionar Dispositivos a Ambientes

Para adicionar dispositivos tivemos alguma dificuldade pois quando era adicionado um dispositivo a um ambiente, esse mesmo era listado no ambiente em causa e nos outros

ambientes todos, e ao eliminar o dispositivo desse ambiente eliminava dos outros ambientes também. Após averiguar a origem do erro foi diagnosticado que a adição de um dispositivo a um ambiente estava mal implementada, a ligação estava a ser feita de muitos para muitos, e devia ser de muitos dispositivos para um ambiente.

5.4 Limitar Acesso de Utilizadores a Ambientes

Foi implementado um bloqueio de utilizados a um ambiente baseado no seu role, pois foi decidido que apenas certos roles deviam estar associados a um ambiente, um escritório não faz sentido ter associado um utilizador com o role de Chef. Assim apenas e possível adicionar utilizadores que tenham roles que sejam permitidos pelo ambiente. Portanto a conexão entre utilizadores e ambientes não e propriamente simples, outro erro logico de filtragem que tivemos foi permitir que o mesmo utilizador estivesse com checkin em diferentes ambientes ao mesmo tempo. Foi adicionado um valor booleano ao utilizador que muda para um(verdadeiro) quando o utilizador faz um checkin, permitindo corrigir o erro.

5.5 Listagens

Cada listagem de seguia uma logica semelhante, a implementação foi relativamente simples.

5.6 Cálculo da fórmula

Para implementar a solução esta foi a parte mais importante pois sem a fórmula toda a aplicação e redundante. Para acertar a fórmula tivemos dificuldade, nomeadamente em agrupar os dados de tabelas diferentes, e acertar as preferências de os utilizadores com as preferências do ambiente. Por fim ficamos satisfeitos com a fórmula que apresentados.

Tipo Ambiente	Peso da preferência do ambiente	Temperatura preferida	Humidade preferida	Luminosidade preferida	Volume preferido	Utilizadores	Pesos do role	Temperatura preferida utilizador	Luminosidade preferida utilizador	Humidade preferida utilizador	Volume preferido utilizador	Temperatura calculada	Humidade calculada	Luminosidade calculada	Volume calculado
Restaurant	0.2	20	50	20	50	regular,chef,admin	0.1, 0.4, 0.45	20, 25, 15	20, 20, 50	20, 20, 20	20, 20, 30	19.8	40.3	20	34.5
Bar	0.1	20	50	50	50	admin,funcionario,funcionario	0.45, 0.2, 0.2	25, 20, 30	50, 50, 50	50, 20, 50	50, 60, 50	24.6	44.8	50	51.7
Home	0.3	20	50	50	50	admin,visitante,visitante	0.45, 0.1, 0.1	15, 20, 30	50, 40, 50	50, 60, 50	50, 30, 60	22.6	49.4	50.6	51.6

Tabela 1 – Tabela de testes

Capítulo 6

6 Conclusões

A construção da plataforma web para a gestão de ambientes e monitoramento de preferências de utilizadores em diferentes cenários representa um avanço significativo no controle e eficiência dos processos internos. Através desta aplicação, foi possível desenvolver uma solução abrangente e integrada, capaz de atender às necessidades específicas da organização. A plataforma permite o registo e atribuição de preferências de ambientes aos utilizadores, respeitando as hierarquias e papéis estabelecidos, garantindo que as condições ambientais sejam adequadas para cada cenário.

A plataforma abrange funcionalidades adicionais, como a gestão de ambientes, utilizadores, papéis (roles) e registos de condições ambientais (records). Essas características permitem uma administração completa e flexível, facilitando a adaptação às necessidades específicas da organização e otimizando os processos relacionados à gestão de ambientes.

No geral a plataforma web desenvolvida apresenta-se como uma solução robusta e eficiente para a gestão de conflitos de preferências de utilizadores no ambiente. Com a implementação, espera-se uma melhoria significativa na organização, proporcionando maior eficiência operacional, redução de custos e tomadas de decisão mais fundamentadas.

6.1 Resultados Alcançados

Durante o desenvolvimento e implantação da plataforma de gestão de conflitos de preferências de utilizadores nos ambientes, diversos resultados foram alcançados. Os principais resultados incluem:

- **Plataforma Funcional:** A plataforma de gestão de ambientes implementada está funcionando de acordo com os requisitos estabelecidos. Os componentes e funcionalidades descritos foram desenvolvidos e integrados com sucesso,

proporcionando uma solução completa para a gestão de ambientes em uma organização.

- **Melhoria na Gestão de Ambientes:** Através do sistema, é possível registrar e monitorar as condições ambientais, atribuí-las a ambientes específicos e monitorar seu uso e preferências. Isso proporciona maior controle e visibilidade sobre os recursos da organização, facilitando a tomada de decisões e otimizando o uso dos ambientes.
- **Aumento da Eficiência e Produtividade:** Com a plataforma em funcionamento, as atividades relacionadas à gestão de ambientes se tornaram mais eficientes e produtivas. Os processos manuais e papelada foram substituídos por um sistema automatizado, reduzindo erros e tempo gasto em tarefas repetitivas. Os colaboradores podem aceder as informações de forma rápida e precisa, facilitando a tomada de decisões e agilizando as operações relacionadas aos ambientes.

6.2 Lições Aprendidas

Durante o desenvolvimento e implantação da plataforma, diversas lições foram aprendidas. Algumas das principais lições aprendidas incluem:

- **Requisitos Claros e Documentados:** A importância de ter requisitos claros e bem documentados desde o início do projeto foi destacada. Isso ajudou a evitar ambiguidades e garantir que a solução desenvolvida atendesse às necessidades da organização.
- **Testes e Validações:** A realização de testes e validações foi fundamental para identificar e corrigir problemas ao longo do desenvolvimento. A abordagem

de desenvolvimento ágil permitiu a adaptação contínua e a melhoria da solução com base nos feedbacks recebidos durante as etapas de teste.

6.3 Melhorias Futuras

Embora a plataforma de gestão de conflitos de preferências tenha sido implementada com sucesso, existem oportunidades de melhoria que podem ser consideradas no futuro:

- **Expansão de Funcionalidades Móveis:** Implementar e melhorar funcionalidades móveis para que os utilizadores possam gerenciar as suas preferências e receber notificações diretamente nos dispositivos móveis, aumentando a conveniência e a acessibilidade.
- **Relatórios e Análises Avançadas:** Desenvolver funcionalidades de relatórios e análises avançadas para fornecer insights detalhados sobre os conflitos de preferências e as soluções aplicadas.
- **Notificações e Alertas:** Implementar notificações e alertas para informar os utilizadores sobre mudanças nas condições ambientais ou eventos importantes relacionados aos ambientes, auxiliando na prevenção de novos conflitos.

6.4 Conclusão

A plataforma de gestão de conflitos de preferências proporcionou uma solução eficiente e centralizada na gestão de condições ambientais. Através do registo, monitoramento e ajustes das condições, a organização ganhou maior controlo, eficiência e produtividade nas suas operações.

A implementação da plataforma envolveu a análise de requisitos, o projeto de arquitetura, o desenvolvimento das funcionalidades, a implantação do sistema e a realização de testes e validações. Aprendemos importantes lições ao longo do processo, contribuindo para a melhoria contínua do projeto.

Com as melhorias futuras consideradas, a plataforma de gestão de conflitos de preferências tem o potencial de se tornar uma ferramenta ainda mais poderosa e abrangente, auxiliando a organização na otimização dos recursos e na melhoria da eficiência operacional.

Bibliografia

- [1] Visual Studio, <https://visualstudio.microsoft.com/> (visitado em 10/11/2023).
- [2] GitLab, <https://about.gitlab.com/> (visitado em 10/11/2023).
- [3] *Erdplus*, <https://erdplus.com/> (visitado em 14/11/2023).
- [4] MySQL Server, <https://github.com/mysql/mysql-server> (visitado em 15/11/2023).
- [5] MYSQL WorkBench, <https://www.mysql.com/products/workbench/> (visitado em 15/11/2023).
- [6] *Node.js*, <https://nodejs.org/pt/> (visitado em 16/01/2023).
- [7] React, <https://react.dev/> (visitado em 18/01/2023).
- [8] JavaScript, <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript> (visitado em 18/11/2023).
- [9] *Stackoverflow javascript questions*,
<https://stackoverflow.com/questions/tagged/javascript/> (visitado em 20/01/2024).
- [10] *Back-End Web Development using JavaScript and node.js*,
<https://www.youtube.com/watch?v=0Jmp5Q3l0xc&list=PL7zl8TDRnbunBKxH8Dmd3xaP7CNeSafSOhttps://pt.stackoverflow.com/questions/tagged/react> (visitado em 01/02/2024).

Apêndice A- Instalação do MySQL Workbench e MySQL Community Server

Para iniciar o desenvolvimento do Projeto começamos por instalar o MySQL Workbench utilizando que instalamos do site oficial. Foi feita uma instalação simples:

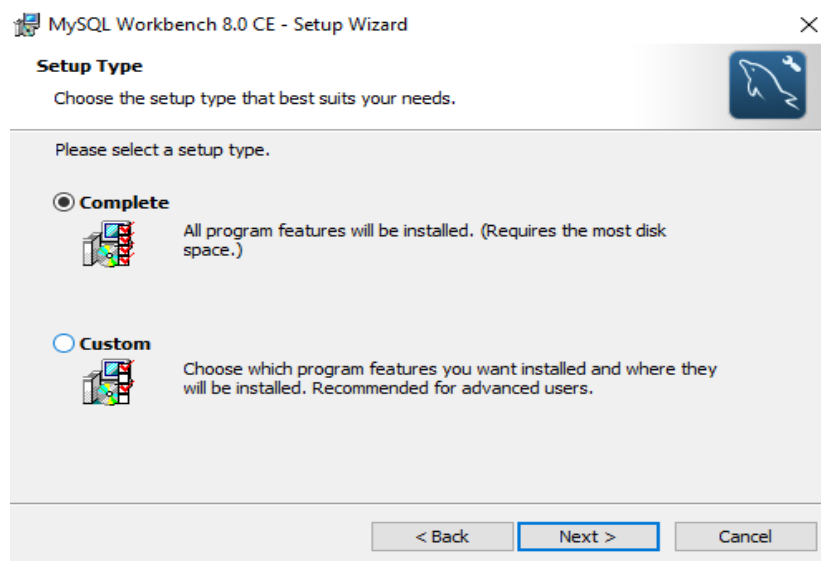


Figura 16- Instalação MySQL Workbench

Para o MySQL Community Server foi feito o mesmo:

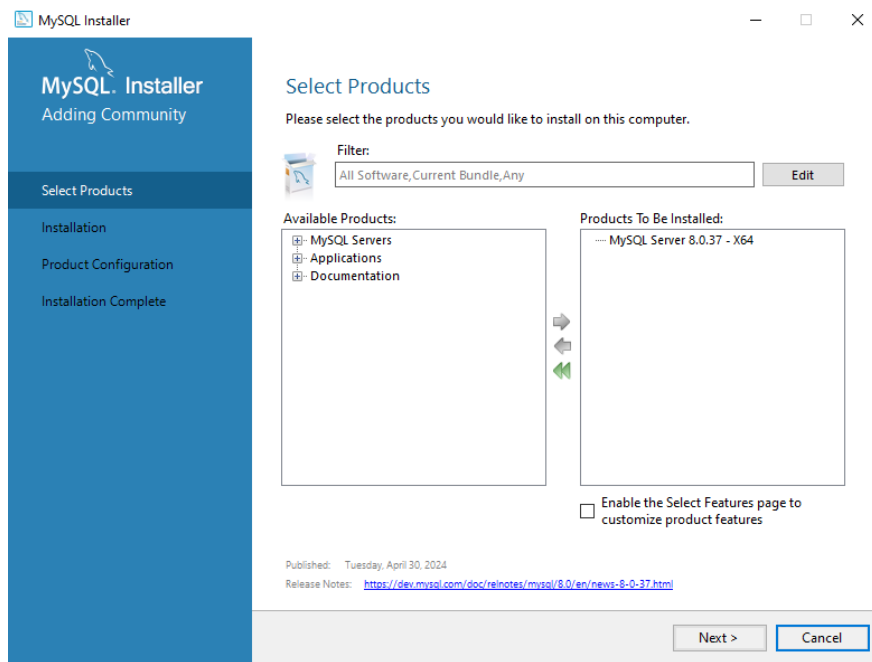


Figura 17- Instalação MySQL

Usamos o MySQLWorkbench com conexão ao MySQL Server para criar e editar a base de dados. Usamos primeiramente o comando “CREATE DATABASE mydb;” e “USE mydb;”.

Estes comandos serviram para criar a base de dados e usar a mesma. Em seguida tivemos que usar comandos para criação de tabelas de forma a estruturar a base de dados com uma estrutura semelhante ao do modelo criado no ERDplus.

Por exemplo:

“CREATE TABLE Usuarios (

ID_usuario INT PRIMARY KEY,

ID_ambiente INT,

Nome_usuario VARCHAR(255),

Outras_informacoes TEXT,

FOREIGN KEY (ID_ambiente) REFERENCES Ambientes(ID_ambiente)

); “Tivemos também de usar o comando” ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY '123456';” Isso foi necessário para permitir a conexão

entre a base de dados MySQL e a aplicação Node.js que utiliza o pacote `mysql` para se comunicar com o banco de dados.

Apêndice B- Instalação do Node.js

Foi feito download do Node.js do site oficial e foi feita uma instalação simples:

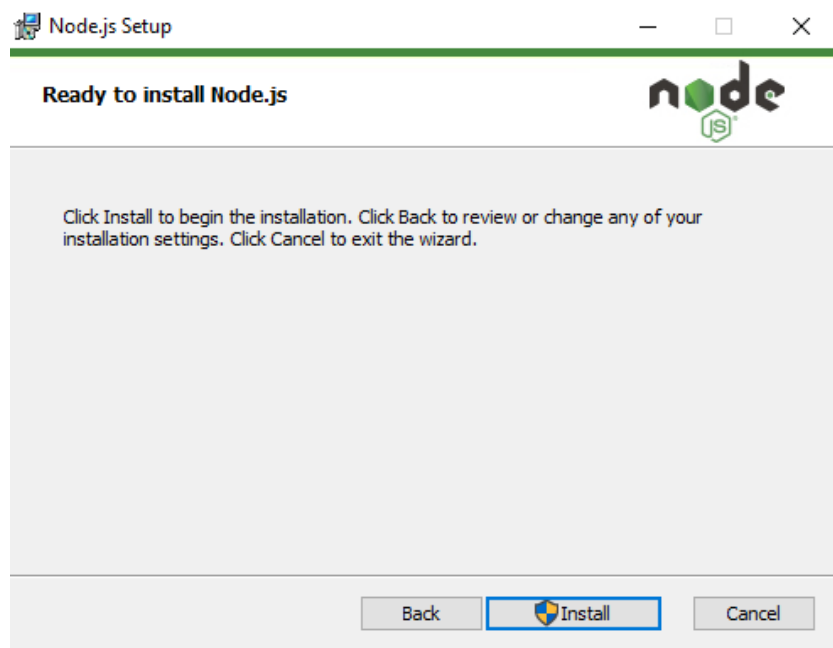
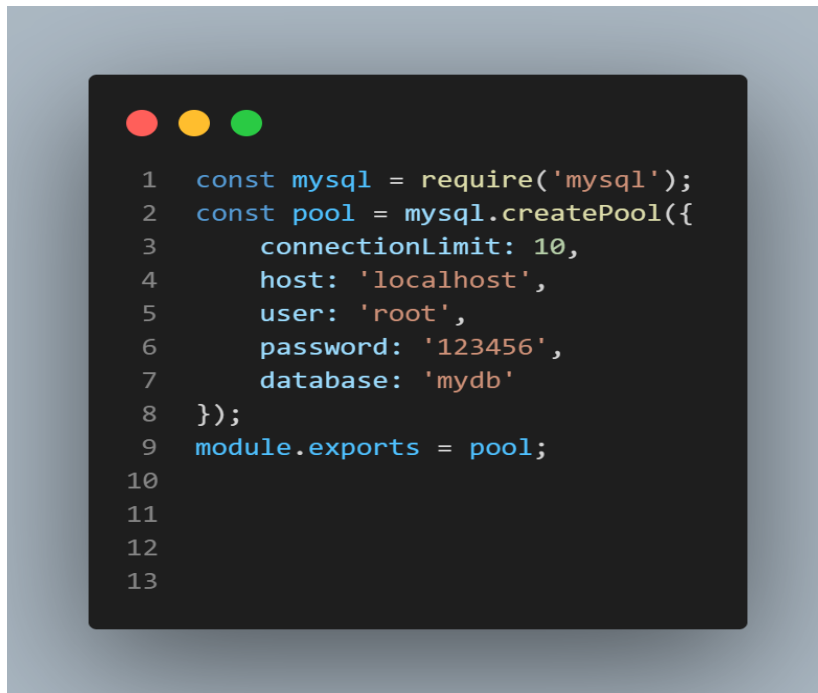


Figura 18- Instalação Node.js

Criamos uma pasta chama servidor-js onde abrimos o terminal do Windows e executamos o comando “npm install express”, e o comando “npm install -g npm@latest” para instalar o Express.js é um framework web popular para Node.js que simplifica o desenvolvimento do backend.

Apêndice C- Configuração da conexão à base de dados

Foi usado este código para estabelecer a ligação entre a base de dados e a aplicação Node.js:

A code editor window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The code is written in a light blue monospace font. It shows the setup of a MySQL connection pool using the 'mysql' module. The code includes line numbers from 1 to 13 on the left side of the editor.

```
1  const mysql = require('mysql');
2  const pool = mysql.createPool({
3      connectionLimit: 10,
4      host: 'localhost',
5      user: 'root',
6      password: '123456',
7      database: 'mydb'
8  });
9  module.exports = pool;
10
11
12
13
```

Figura 19- Ligação com a base de dados

Para permitir a interação com a base de dados através de várias operações CRUD, foi implementado um servidor utilizando o framework Express. Este servidor define múltiplas rotas que correspondem a diferentes recursos e operações na base de dados:

```
1  const express = require('express');
2  const app = express();
3  const bodyParser = require('body-parser');
4  const pool = require('./cruds/db');
5  const cors = require('cors');
6
7  app.use(bodyParser.json());
8
9  const usersRoutes = require('./cruds/usuarios');
10 const ambientesRoutes = require('./cruds/ambientes');
11 const dispositivosRoutes = require('./cruds/dispositivos');
12 const checkinRoutes = require('./cruds/checkin');
13 const roleRoutes = require('./cruds/roles');
14 const rolesDoAmbienteRoutes = require('./cruds/rolesDoAmbiente');
15 const preferencias = require('./cruds/preferencias');
16 const registros = require('./cruds/registros')
17
18 app.use(cors());
19 app.use('/usuarios', usersRoutes);
20 app.use('/ambientes', ambientesRoutes);
21 app.use('/dispositivos', dispositivosRoutes);
22 app.use('/checkins', checkinRoutes);
23 app.use('/roles', roleRoutes);
24 app.use('/rolesDoAmbiente', rolesDoAmbienteRoutes);
25 app.use('/preferencias', preferencias);
26 app.use('/registros', registros);
27
28 const PORT = process.env.PORT || 8080;
29
30 app.listen(PORT, () => {
31   console.log(`Server is running on port ${PORT}`);
32 });
33
34
35
```

Figura 20- Múltiplas Rotas do servidor

Apêndice D– Configuração da API



```
1  const express = require('express');
2  const app = express();
3  const bodyParser = require('body-parser');
4  const pool = require('./cruds/db');
5  const cors = require('cors');
6
7  app.use(bodyParser.json());
8
9  const usersRoutes = require('./cruds/usuarios');
10 const ambientesRoutes = require('./cruds/ambientes');
11 const dispositivosRoutes = require('./cruds/dispositivos');
12 const checkinRoutes = require('./cruds/checkin');
13 const roleRoutes = require('./cruds/roles');
14 const rolesDoAmbienteRoutes = require('./cruds/rolesDoAmbiente');
15 const preferencias = require('./cruds/preferencias');
16 const registros = require('./cruds/registros')
17
18 app.use(cors());
19 app.use('/usuarios', usersRoutes);
20 app.use('/ambientes', ambientesRoutes);
21 app.use('/dispositivos', dispositivosRoutes);
22 app.use('/checkins', checkinRoutes);
23 app.use('/roles', roleRoutes);
24 app.use('/rolesDoAmbiente', rolesDoAmbienteRoutes);
25 app.use('/preferencias', preferencias);
26 app.use('/registros', registros);
27
28 const PORT = process.env.PORT || 8080;
29
30 app.listen(PORT, () => {
31   console.log(`Server is running on port ${PORT}`);
32 });
33
34
35
```

Figura 21- API da Aplicação

```

1  const API_URL = 'http://localhost:8080/ambientes';
2
3
4  // Fetch all ambientes
5  export const fetchAmbientes = async () => {
6    try {
7      const response = await fetch(API_URL, {
8        method: 'GET',
9        headers: {
10          'Content-Type': 'application/json',
11        },
12      });
13      const responseData = await response.json();
14      return responseData;
15    } catch (error) {
16      console.error('Error fetching ambientes:', error);
17      throw new Error('Failed to fetch ambientes');
18    }
19  };
20
21  // DELETE AMBIENTE
22  export const deleteAmbiente = async (id) => {
23    try {
24      const response = await fetch(`${API_URL}/${id}`, {
25        method: 'DELETE',
26        headers: {
27          'Content-Type': 'application/json',
28        },
29      });
30      return { success: true };
31    } catch (error) {
32      console.error('Error deleting ambiente:', error);
33      throw new Error('Failed to delete ambiente');
34    }
35  };
36
37  // CREATE AMBIENTE
38  export const createAmbiente = async (ambiente) => {
39    try {
40      const response = await fetch(API_URL, {
41        method: 'POST',
42        headers: {
43          'Content-Type': 'application/json',
44        },
45        body: JSON.stringify(ambiente),
46      });
47      const responseData = await response.json();
48      return responseData;
49    } catch (error) {
50      console.error('Error creating ambiente:', error);
51      throw new Error('Failed to create ambiente');
52    }
53  };
54
55  // UPDATE AMBIENTE
56  export const updateAmbiente = async (id, ambiente) => {
57    try {
58      const response = await fetch(`${API_URL}/${id}`, {
59        method: 'PUT',
60        headers: {
61          'Content-Type': 'application/json',
62        },
63        body: JSON.stringify(ambiente),
64      });
65      const responseData = await response.json();
66      return responseData;
67    } catch (error) {
68      console.error('Error updating ambiente:', error);
69      throw new Error('Failed to update ambiente');
70    }
71  };
72
73  // Fetch temp for device
74  export const fetchCalculatedTemp = async (ambienteId) => {
75    try {
76      const response = await fetch(`${API_URL}/${ambienteId}`, {
77        method: 'GET',
78        headers: {
79          'Content-Type': 'application/json',
80        },
81      });
82      const responseData = await response.json();
83      return responseData;
84    } catch (error) {
85      console.error('Error fetching ambiente preference:', error);
86      throw new Error('Failed to fetch ambiente preference');
87    }
88  };
89
90
91

```

Figura 22- API da página dos Ambientes

```

1  const API_URL = 'http://localhost:8080/preferencias';
2
3  // READ
4  export const fetchAllPreferencias = async () => {
5    try {
6      const response = await fetch(API_URL, {
7        method: 'GET',
8        headers: {
9          'Content-Type': 'application/json',
10        },
11      });
12      const responseData = await response.json();
13      return responseData;
14    } catch (error) {
15      console.error('Error fetching preferencias:', error);
16      throw new Error('Failed to fetch preferencias');
17    }
18  };
19
20  // CREATE
21  export const createPreferencia = async (preferencia) => {
22    try {
23      const response = await fetch(API_URL, {
24        method: 'POST',
25        headers: {
26          'Content-Type': 'application/json',
27        },
28        body: JSON.stringify(preferencia),
29      });
30      const responseData = await response.json();
31      return responseData;
32    } catch (error) {
33      console.error('Error creating preferencia:', error);
34      throw new Error('Failed to create preferencia');
35    }
36  };
37
38  // UPDATE
39  export const updatePreferencia = async (id, preferencia) => {
40    try {
41      const response = await fetch(`${API_URL}/${id}`, {
42        method: 'PUT',
43        headers: {
44          'Content-Type': 'application/json',
45        },
46        body: JSON.stringify(preferencia),
47      });
48      const responseData = await response.json();
49      return responseData;
50    } catch (error) {
51      console.error('Error updating preferencia:', error);
52      throw new Error('Failed to update preferencia');
53    }
54  };
55
56  // READ by ID
57  export const fetchPreferencias = async (userId) => {
58    try {
59      const response = await fetch(`${API_URL}/preferencias/${userId}`, {
60        method: 'GET',
61        headers: {
62          'Content-Type': 'application/json',
63        },
64      });
65    } catch (error) {
66      console.error('Error fetching preferencias:', error);
67      throw new Error('Failed to fetch preferencias');
68    }
69  };
70
71  };
72

```

Figura 23- API da página das Preferências

```

1  const API_URL = 'http://localhost:8080/dispositivos';
2
3  // READ ALL DISPOSITIVOS
4  export const fetchDispositivos = async () => {
5    try {
6      const response = await fetch(API_URL, {
7        method: 'GET',
8        headers: {
9          'Content-Type': 'application/json',
10        },
11      });
12
13      const responseData = await response.json();
14      return responseData;
15    } catch (error) {
16      console.error('Error fetching dispositivos:', error);
17      throw new Error('Failed to fetch dispositivos');
18    }
19  };
20
21  // CREATE a new dispositivo
22  export const createDispositivo = async (formData) => {
23    try {
24      const response = await fetch(API_URL, {
25        method: 'POST',
26        headers: {
27          'Content-Type': 'application/json',
28        },
29        body: JSON.stringify(formData),
30      });
31      const data = await response.json();
32      return data;
33    } catch (error) {
34      throw new Error('Error adding dispositivo:', error);
35    }
36  };
37
38  // UPDATE DISPOSITIVO
39  export const updateDispositivo = async (dispositivo) => {
40    const response = await fetch(`${API_URL}/${dispositivo.ID_dispositivo}`, {
41      method: 'PUT',
42      headers: {
43        'Content-Type': 'application/json',
44      },
45      body: JSON.stringify(dispositivo),
46    });
47    return await response.json();
48  };
49  // DELETE DISPOSITIVO
50  export const deleteDispositivo = async (id) => {
51    try {
52      const response = await fetch(`${API_URL}/${id}`, {
53        method: 'DELETE',
54        headers: {
55          'Content-Type': 'application/json',
56        },
57      });
58      return { success: true };
59    } catch (error) {
60      console.error('Error deleting dispositivo:', error);
61      throw new Error('Failed to delete dispositivo');
62    }
63  };
64
65  // Fetch dispositivos by ID_ambiente
66  export const fetchDispositivosByAmbiente = async (ID_ambiente) => {
67    try {
68      const response = await fetch(`${API_URL}/ambientes/${ID_ambiente}`, {
69        method: 'GET',
70        headers: {
71          'Content-Type': 'application/json',
72        },
73      });
74      const responseData = await response.json();
75      return responseData;
76    } catch (error) {
77      console.error('Error fetching dispositivos:', error);
78      throw new Error('Failed to fetch dispositivos');
79    }
80  };

```

Figura 24- API da página dos Dispositivos

```
1  const API_URL = 'http://localhost:8080/checkins';
2
3  // Check in a user
4  export const checkInUser = async (ID_usuario, ID_ambiente) => {
5    const response = await fetch(`${API_URL}/checkin`, {
6      method: 'POST',
7      headers: {
8        'Content-Type': 'application/json',
9      },
10     body: JSON.stringify({ ID_usuario, ID_ambiente }),
11   });
12   return await response.json();
13 };
14
15 // Check out a user
16 export const checkOutUser = async (ID_usuario, ID_ambiente) => {
17   const response = await fetch(`${API_URL}/checkout`, {
18     method: 'POST',
19     headers: {
20       'Content-Type': 'application/json',
21     },
22     body: JSON.stringify({ ID_usuario, ID_ambiente }),
23   });
24   return await response.json();
25 };
26
27 // Fetch all users checked into a specific environment
28 export const fetchCheckedInUsers = async (ID_ambiente) => {
29   const response = await fetch(`${API_URL}/users/${ID_ambiente}`, {
30     method: 'GET',
31     headers: {
32       'Content-Type': 'application/json',
33     },
34   });
35
36   return await response.json();
37 };
38
```

Figura 25- API da página dos Checkins



```
1  const API_URL = 'http://localhost:8080/registros';
2
3  // Fetch all registros
4  export const fetchAllRegistros = async () => {
5    try {
6      const response = await fetch(API_URL, {
7        method: 'GET',
8        headers: {
9          'Content-Type': 'application/json',
10        },
11      });
12      const responseData = await response.json();
13      return responseData;
14    } catch (error) {
15      console.error('Error fetching registros:', error);
16      throw new Error('Failed to fetch registros');
17    }
18  };
19
20  // Fetch a single registro by ID
21  export const fetchRegistroById = async (ID_registro) => {
22    const response = await fetch(`${API_URL}/${ID_registro}`, {
23      method: 'GET',
24      headers: {
25        'Content-Type': 'application/json',
26      },
27    });
28    return await response.json();
29  };
30
31  // Delete a registro by ID
32  export const deleteRegistroById = async (ID_registro) => {
33    const response = await fetch(`${API_URL}/${ID_registro}`, {
34      method: 'DELETE',
35      headers: {
36        'Content-Type': 'application/json',
37      },
38    });
39    return await response.json();
40  };
```

Figura 26- API da página dos Registros


```

1  const API_URL = 'http://localhost:8080/roles';
2
3  //create
4  export const createRole = async (userData) => {
5    const response = await fetch(API_URL, {
6      method: 'POST',
7      headers: {
8        'Content-Type': 'application/json',
9      },
10     body: JSON.stringify(userData),
11   });
12   return await response.json();
13 };
14
15
16 // READ ALL ROLES
17 export const fetchRoles = async () => {
18   try {
19     const response = await fetch(API_URL, {
20       method: 'GET',
21       headers: {
22         'Content-Type': 'application/json',
23       },
24     });
25     const responseData = await response.json();
26
27     // Filter the responseData to include only specific data
28     const filteredData = responseData.map(role => ({
29       ...role,
30       // Access the 'role_name' field from the role object
31       role_name: role.role_name
32     }));
33
34     return filteredData;
35   } catch (error) {
36     console.error('Error fetching roles:', error);
37     throw new Error('Failed to fetch roles');
38   }
39 };
40
41 //update
42 export const updateRole = async (id, role) => {
43   try {
44     const response = await fetch(`${API_URL}/${id}`, {
45       method: 'PUT',
46       headers: {
47         'Content-Type': 'application/json',
48       },
49       body: JSON.stringify(role),
50     });
51     const responseData = await response.json();
52     return responseData;
53   } catch (error) {
54     console.error('Error updating role:', error);
55     throw new Error('Failed to update role');
56   }
57 };
58
59 //delete
60 export const deleteRole = async (id) => {
61   const response = await fetch(`${API_URL}/${id}`, {
62     method: 'DELETE',
63   });
64   return await response.json();
65 };
66
67
68

```

Figura 27- API da página dos Roles

```

1  const API_URL = 'http://localhost:8080/rolesDoAmbiente';
2
3  //ADD
4  export const addRoleToAmbiente = async (ID_role, ID_ambiente) => {
5    const response = await fetch(`${API_URL}/add`, {
6      method: 'POST',
7      headers: {
8        'Content-Type': 'application/json',
9      },
10     body: JSON.stringify({ ID_role, ID_ambiente }),
11   });
12
13   return await response.json();
14 };
15
16 // READ ALL
17 export const fetchRolesDoAmbiente = async (ID_ambiente) => {
18   try {
19     const response = await fetch(`${API_URL}/${ID_ambiente}`, {
20       method: 'GET',
21       headers: {
22         'Content-Type': 'application/json',
23       },
24     });
25     const responseData = await response.json();
26     // Filter the responseData to include only specific data
27     const filteredData = responseData.map(role => ({
28       ...role,
29       // Access the 'Tipo_ambiente' field from the role object
30       role_name: role.role_name
31     }));
32     return filteredData;
33   } catch (error) {
34     console.error('Error fetching ambientes:', error);
35     throw new Error('Failed to fetch ambientes');
36   }
37 };
38
39 // DELETE
40 export const deleteRoleDoAmbiente = async (id) => {
41   try {
42     const response = await fetch(`${API_URL}/${id}`, {
43       method: 'DELETE',
44       headers: {
45         'Content-Type': 'application/json',
46       },
47     });
48     return { success: true };
49   } catch (error) {
50     console.error('Error deleting role:', error);
51     throw new Error('Failed to delete role');
52   }
53 };
54
55 // UPDATE
56 export const updateAmbiente = async (id, role) => {
57   try {
58     const response = await fetch(`${API_URL}/${id}`, {
59       method: 'PUT',
60       headers: {
61         'Content-Type': 'application/json',
62       },
63       body: JSON.stringify(role),
64     });
65     const responseData = await response.json();
66     return responseData;
67   } catch (error) {
68     console.error('Error updating role:', error);
69     throw new Error('Failed to update role');
70   }
71 };
72 };
73

```

Figura 28- API da página dos Roles Do Ambiente

```

1  const API_URL = 'http://localhost:8080/usuarios';
2
3
4  //create USER
5  export const createUser = async (userData) => {
6    const response = await fetch(API_URL, {
7      method: 'POST',
8      headers: {
9        'Content-Type': 'application/json',
10     },
11     body: JSON.stringify(userData),
12   });
13   if (!response.ok) {
14     throw new Error('Failed to create user');
15   }
16   return await response.json();
17 };
18
19
20 //READ ALL USERS
21 export const fetchUsers = async () => {
22   const response = await fetch(API_URL, {
23     method: 'GET',
24     headers: {
25       'Content-Type': 'application/json',
26     },
27   });
28   if (!response.ok) {
29     throw new Error('Failed to fetch users');
30   }
31   const responseData = await response.json();
32   // Assuming the response data is an array of user objects
33   // Each user object should contain an 'ID_usuario' and 'role_name' field
34   return responseData.map(user => ({
35     ...user,
36     ID_usuario: user.ID_usuario,
37     role_name: user.role_name,
38     valor_preferencia: user.valor_preferencia // If this field is included in the response
39   }));
40 };
41
42 //DELETE USER
43 export const deleteUser = async (userId) => {
44   const url = `${API_URL}/${userId}`;
45   const response = await fetch(url, {
46     method: 'DELETE',
47   });
48   if (!response.ok) {
49     throw new Error('Failed to delete user');
50   }
51 };
52
53 //update USER AND PREFERENCIAS
54 export const updateUser = async (id, data) => {
55   try {
56     const response = await fetch(`${API_URL}/${id}`, {
57       method: 'PUT',
58       headers: {
59         'Content-Type': 'application/json',
60       },
61       body: JSON.stringify(data),
62     });
63     if (!response.ok) {
64       throw new Error('Failed to update user');
65     }
66     return await response.json();
67   } catch (error) {
68     console.error('Error updating user:', error);
69     throw error;
70   }
71 };
72
73 export const fetchUsersWithAllowedRoles = async (ambienteId) => {
74   const response = await fetch(`${API_URL}/allowedUsers/${ambienteId}`, {
75     method: 'GET',
76     headers: {
77       'Content-Type': 'application/json',
78     },
79   });
80   if (!response.ok) {
81     throw new Error('Failed to fetch users with allowed roles');
82   }
83   return await response.json();
84 };
85
86 // READ USER WITH PREFERENCES BY ID
87 export const fetchUserComPreferencias = async (userId) => {
88   const response = await fetch(`${API_URL}/${userId}`, {
89     method: 'GET',
90     headers: {
91       'Content-Type': 'application/json',
92     },
93   });
94   if (!response.ok) {
95     throw new Error('Failed to fetch user with preferences');
96   }
97   const responseData = await response.json();
98   return responseData;
99 };
100
101

```

Figura 29- API da página dos Utilizadores

Apêndice E– Configuração dos Componentes

```
1 import React, { useState } from 'react';
2 import { createDispositivo } from '../api/dispositivos';
3
4 const AddDispositivoModal = ({ ID_ambiente, isOpen, onClose, onAdd }) => {
5   const [formState, setFormState] = useState({
6     Tipo_dispositivo: '',
7     temperature: 0,
8     amperes: 0,
9     decibels: 0,
10    volume_percentage: 0,
11    estado_dispositivo: 0,
12    ID_ambiente: ID_ambiente,
13  });
14
15  const handleChange = (e) => {
16    const { name, value } = e.target;
17    setFormState(prevState => ({
18      ...prevState,
19      [name]: value,
20    }));
21  };
22
23  const handleSubmit = async (e) => {
24    e.preventDefault();
25    try {
26      await createDispositivo(formState); // Pass formState directly
27      onAdd();
28      onClose();
29    } catch (error) {
30      console.error('Error adding dispositivo:', error);
31    }
32  };
33
34  if (!isOpen) return null;
35
36  return (
37    <div className="modal">
38      <div className="modal-content">
39        <h3>Add Device</h3>
40        <form onSubmit={handleSubmit}>
41          <div>
42            <label>
43              Type:
44              <input type="text" name="Tipo dispositivo" value={formState.Tipo_dispositivo} onChange={handleChange} required />
45            </label>
46          </div>
47          <div>
48            <label>
49              Temperature Control:
50              <select name="temperature" value={formState.temperature} onChange={handleChange}>
51                <option value={0}>No Control</option>
52                <option value={1}>Has Control</option>
53              </select>
54            </label>
55          </div>
56          <div>
57            <label>
58              Amperes Control:
59              <select name="amperes" value={formState.amperes} onChange={handleChange}>
60                <option value={0}>No Control</option>
61                <option value={1}>Has Control</option>
62              </select>
63            </label>
64          </div>
65          <div>
66            <label>
67              Decibels Control:
68              <select name="decibels" value={formState.decibels} onChange={handleChange}>
69                <option value={0}>No Control</option>
70                <option value={1}>Has Control</option>
71              </select>
72            </label>
73          </div>
74          <div>
75            <label>
76              Volume Control:
77              <select name="volume_percentage" value={formState.volume_percentage} onChange={handleChange}>
78                <option value={0}>No Control</option>
79                <option value={1}>Has Control</option>
80              </select>
81            </label>
82          </div>
83          <div>
84            <label>
85              State (ON/OFF):
86              <select name="estado_dispositivo" value={formState.estado_dispositivo} onChange={handleChange}>
87                <option value={0}>OFF</option>
88                <option value={1}>ON</option>
89              </select>
90            </label>
91          </div>
92          <button type="submit">Add Device</button>
93          <button type="button" onClick={onClose}>Cancel</button>
94        </form>
95      </div>
96    </div>
97  );
98 };
99
100 export default AddDispositivoModal;
101
```

Figura 30- Componente do AddDispositivoModal

```

1 import React, { useState } from 'react';
2 import { createDispositivo } from '../api/dispositivos';
3
4 const AddDispositivoModal = ({ ID_ambiente, isOpen, onClose, onAdd }) => {
5   const [formState, setFormState] = useState({
6     Tipo_dispositivo: '',
7     temperature: 0,
8     amperes: 0,
9     decibels: 0,
10    volume_percentage: 0,
11    Estado_dispositivo: 0,
12    ID_ambiente: ID_ambiente,
13  });
14
15   const handleChange = (e) => {
16     const { name, value } = e.target;
17     setFormState(prevState => ({
18       ...prevState,
19       [name]: value,
20     }));
21   };
22
23   const handleSubmit = async (e) => {
24     e.preventDefault();
25     try {
26       await createDispositivo(formState); // Pass formState directly
27       onAdd();
28       onClose();
29     } catch (error) {
30       console.error('Error adding dispositivo:', error);
31     }
32   };
33
34   if (!isOpen) return null;
35
36   return (
37     <div className="modal">
38       <div className="modal-content">
39         <h3>Add Device:</h3>
40         <form onSubmit={handleSubmit}>
41           <div>
42             <label>
43               Type:
44               <input type="text" name="Tipo_dispositivo" value={formState.Tipo_dispositivo} onChange={handleChange} required />
45             </label>
46           </div>
47           <div>
48             <label>
49               Temperature Control:
50               <select name="temperature" value={formState.temperature} onChange={handleChange}>
51                 <option value={0}>No Control</option>
52                 <option value={1}>Has Control</option>
53               </select>
54             </label>
55           </div>
56           <div>
57             <label>
58               Amperes Control:
59               <select name="amperes" value={formState.amperes} onChange={handleChange}>
60                 <option value={0}>No Control</option>
61                 <option value={1}>Has Control</option>
62               </select>
63             </label>
64           </div>
65           <div>
66             <label>
67               Decibels Control:
68               <select name="decibels" value={formState.decibels} onChange={handleChange}>
69                 <option value={0}>No Control</option>
70                 <option value={1}>Has Control</option>
71               </select>
72             </label>
73           </div>
74           <div>
75             <label>
76               Volume Control:
77               <select name="volume_percentage" value={formState.volume_percentage} onChange={handleChange}>
78                 <option value={0}>No Control</option>
79                 <option value={1}>Has Control</option>
80               </select>
81             </label>
82           </div>
83           <div>
84             <label>
85               State (ON/OFF):
86               <select name="Estado_dispositivo" value={formState.Estado_dispositivo} onChange={handleChange}>
87                 <option value={0}>OFF</option>
88                 <option value={1}>ON</option>
89               </select>
90             </label>
91           </div>
92           <button type="submit">Add Device</button>
93           <button type="button" onClick={onClose}>Cancel</button>
94         </form>
95       </div>
96     </div>
97   );
98 }
99
100 export default AddDispositivoModal;
101

```

Figura 31- Componente do EditDispositivoModal

[illegible]

Figura 32- Componente do AddUserModal


```

1  /* AddUserModal.css */
2
3  .modal-overlay {
4    position: fixed;
5    top: 0;
6    left: 0;
7    width: 100%;
8    height: 100%;
9    background-color: rgba(0, 0, 0, 0.5);
10   display: flex;
11   justify-content: center;
12   align-items: center;
13   z-index: 1000;
14 }
15
16 .modal-content {
17   background-color: white;
18   padding: 20px;
19   border-radius: 8px;
20   position: relative;
21 }
22
23 .close-button {
24   position: absolute;
25   top: 0px;
26   right: 0px;
27   background: red;
28   border: none; /* Remove border */
29   border-radius: 25%; /* Make the button round */
30   font-size: 15px;
31   cursor: pointer;
32 }
33
34 .modal {
35   display: flex;
36   justify-content: center;
37   align-items: center;
38   position: fixed;
39   z-index: 1;
40   left: 0;
41   top: 0;
42   width: 100%;
43   height: 100%;
44   overflow: auto;
45   background-color: rgb(0, 0, 0);
46   background-color: rgba(0, 0, 0, 0.4);
47 }
48
49 .modal-content {
50   background-color: #f4f4f4;
51   margin: auto;
52   padding: 20px;
53   border: 1px solid #888;
54   width: 80%;
55   max-width: 500px;
56   border-radius: 8px;
57   box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
58 }
59
60 .close {
61   color: #aaa;
62   float: right;
63   font-size: 28px;
64   font-weight: bold;
65 }
66
67 .close:hover,
68 .close:focus {
69   color: black;
70   text-decoration: none;
71   cursor: pointer;
72 }
73
74 .button {
75   padding: 10px 20px;
76   border: none;
77   border-radius: 4px;
78   background-color: #007bff;
79   color: white;
80   cursor: pointer;
81 }
82
83 .button:hover {
84   background-color: #0056b3;
85 }
86
87 .error-icon {
88   bottom: 10%;
89   position: relative;
90   display: inline;
91   margin-left: 10px; /* Adjust as needed for spacing */
92   background-color: rgb(72, 24, 24);
93 }
94
95 .rectangle-with-triangle {
96   position: absolute;
97   left: 10%;
98   top: 0%;
99   width: 20px; /* Width of the rectangle */
100  height: 20px; /* Height of the rectangle */
101  background-color: red; /* Color of the rectangle */
102  clip-path: polygon(0% 0%, 100% 0%, 100% 80%, 50% 100%, 0% 80%); /* Triangle shape */
103 }
104
105
106
107

```

Figura 34- Componente do AddUserModalCss


```

1 import React, { useEffect } from 'react';
2 import { useForm } from 'react-hook-form';
3 import { yupResolver } from '@hookform/resolvers/yup';
4 import * as yup from 'yup';
5 import { createRole, updateRole } from '../api/roles';
6 import './Roles.css';
7
8 const RoleForm = ({ role, onCreate, onEdit, onClose }) => {
9   const validationSchema = yup.object().shape({
10     role_name: yup.string().required('Role Name is required'),
11     role_weight: yup.number()
12       .typeError('Role Weight must be a number')
13       .min(0, 'Role Weight must be at least 0')
14       .max(1, 'Role Weight must be at most 1')
15       .required('Role Weight is required')
16   });
17
18   const {
19     register,
20     handleSubmit,
21     setValue,
22     formState: { errors }
23   } = useForm({
24     resolver: yupResolver(validationSchema)
25   });
26
27   useEffect(() => {
28     if (role) {
29       setValue('role_name', role.role_name);
30       setValue('role_weight', role.role_weight);
31     }
32   }, [role, setValue]);
33
34   const onSubmit = async (data) => {
35     try {
36       if (role) {
37         await updateRole(role.id, data);
38         onEdit({ ...role, ...data });
39       } else {
40         const newRole = await createRole(data);
41         onCreate(newRole);
42       }
43       onClose(); // Close the modal after submitting
44     } catch (error) {
45       console.error('Error updating role:', error);
46     }
47   });
48
49   return (
50     <div className="modal">
51       <div className="modal-content">
52         <span className="close" onClick={onClose}>&times;</span>
53         <h2>{role ? 'Edit Role' : 'Add Role'}</h2>
54         <form onSubmit={handleSubmit(onSubmit)}>
55           <div>
56             <label>Role Name:</label>
57             <input
58               type="text"
59               {...register('role_name')}
60             />
61             {errors.role_name && <p>{errors.role_name.message}</p>}
62           </div>
63           <div>
64             <label>Role Weight:</label>
65             <input
66               type="text"
67               {...register('role_weight')}
68             />
69             {errors.role_weight && <p>{errors.role_weight.message}</p>}
70           </div>
71           <button type="submit">{role ? 'Edit Role' : 'Add Role'}</button>
72         </form>
73       </div>
74     </div>
75   );
76 };
77
78 export default RoleForm;
79

```

Figura 35- Componente do RoleForm

```

1 import React, { useEffect, useState } from 'react';
2 import { useForm } from 'react-hook-form';
3 import * as Yup from 'yup';
4 import { yupResolver } from '@hookform/resolvers/yup';
5 import { fetchAmbientes } from '../api/ambientes';
6 import { fetchRoles } from '../api/roles';
7 import './Roles.css';
8
9 const EditRoleModal = ({ role, onClose, onSave }) => {
10   const validationSchema = Yup.object().shape({
11     role_name: Yup.string().required('Role Name is required'),
12     role_weight: Yup.number()
13       .typeError('Role Weight must be a number')
14       .min(0, 'Role Weight must be at least 0')
15       .max(1, 'Role Weight must be at most 1')
16       .required('Role Weight is required'),
17   });
18
19   const {
20     register,
21     handleSubmit,
22     setValue,
23     formState: { errors }
24   } = useForm({
25     resolver: yupResolver(validationSchema),
26     defaultValues: {
27       role_name: role.role_name,
28       role_weight: role.role_weight,
29     }
30   });
31
32   useEffect(() => {
33     const fetchData = async () => {
34       try {
35         const ambientesData = await fetchAmbientes();
36         const rolesData = await fetchRoles();
37         setAmbientes(ambientesData);
38         setRoles(rolesData);
39       } catch (error) {
40         console.error('Error fetching data:', error);
41       }
42     };
43
44     fetchData();
45   }, []);
46
47   const [ambientes, setAmbientes] = useState([]);
48   const [roles, setRoles] = useState([]);
49
50   const onSubmit = (data) => {
51     onSave({ ...role, ...data });
52     onClose();
53   };
54
55   return (
56     <div className="modal">
57       <div className="modal-content">
58         <span className="close" onClick={onClose}>&times;</span>
59         <h2>Edit Role</h2>
60         <form onSubmit={handleSubmit(onSubmit)}>
61           <div>
62             <label>Role Name:</label>
63             <input
64               type="text"
65               name="role_name"
66               {...register('role_name')}
67             />
68             {errors.role_name && <p>{errors.role_name.message}</p>}
69           </div>
70           <div>
71             <label>Role Weight:</label>
72             <input
73               type="text"
74               name="role_weight"
75               {...register('role_weight')}
76             />
77             {errors.role_weight && <p>{errors.role_weight.message}</p>}
78           </div>
79           <button type="submit">Save</button>
80         </form>
81       </div>
82     </div>
83   );
84 };
85
86 export default EditRoleModal;
87

```

Figura 36- Componente do EditRoleModal

```

1  /* Styles for the role list container */
2  .role-list {
3      padding: 20px;
4      background-color: #e0e0e0; /* Similar background color */
5      border-radius: 8px;
6      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
7  }
8
9  /* Styles for the list items */
10 .role-item {
11     margin-bottom: 10px;
12     padding: 10px;
13     background-color: #fff;
14     border-radius: 4px;
15     box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
16     display: flex;
17     align-items: center;
18     justify-content: flex-start;
19 }
20
21 /* Styles for the role info */
22 .role-info {
23     margin-right: 10px;
24     width: 1000px;
25 }
26
27 /* Styles for the button group */
28 .button-group {
29     display: flex;
30     gap: 5px;
31     justify-content: flex-end; /* Align to the right */
32 }
33
34 /* Styles for the buttons */
35 .button {
36     padding: 8px 16px;
37     border: none;
38     border-radius: 4px;
39     background-color: #007bff;
40     color: #fff;
41     cursor: pointer;
42 }
43 .button:hover {
44     background-color: #0056b3;
45 }
46 .role-list-header {
47     display: flex;
48     align-items: center;
49 }
50
51 .role-list-header h2 {
52     position: relative;
53     left: 40px;
54 }
55
56 .button_add_role {
57     position: sticky;
58     padding: 8px 16px;
59     border: none;
60     border-radius: 4px;
61     background-color: #007bff;
62     color: #fff;
63     cursor: pointer;
64     left: 1060px;
65     top: 0;
66 }
67
68 .button_add_role:hover {
69     background-color: #0056b3;
70 }
71
72 /* Styles for the headers */
73 h2 {
74     font-size: 24px; /* Adjusted the font size for consistency */
75     margin-bottom: 20px;
76     color: #333;
77 }
78

```

Figura 37- Componente do RoleCss

```

1 import React, { useState } from 'react';
2 import { createDispositivo } from '../api/dispositivos';
3
4 const AddDispositivoModal = ({ ID_ambiente, isOpen, onClose, onAdd }) => {
5   const [formState, setFormState] = useState({
6     Tipo_dispositivo: '',
7     temperature: 0,
8     amperes: 0,
9     decibels: 0,
10    volume_percentage: 0,
11    estado_dispositivo: 0,
12    ID_ambiente: ID_ambiente,
13  });
14
15  const handleChange = (e) => {
16    const { name, value } = e.target;
17    setFormState((prevState) => ({
18      ...prevState,
19      [name]: value,
20    }));
21  };
22
23  const handleSubmit = async (e) => {
24    e.preventDefault();
25    try {
26      await createDispositivo(formState); // Pass formState directly
27      onAdd();
28      onClose();
29    } catch (error) {
30      console.error('Error adding dispositivo:', error);
31    }
32  };
33
34  if (!isOpen) return null;
35
36  return (
37    <div className="modal">
38      <div className="modal-content">
39        <h3>Add Device</h3>
40        <form onSubmit={handleSubmit}>
41          <div>
42            <label>
43              Type:
44              <input type="text" name="Tipo_dispositivo" value={formState.Tipo_dispositivo} onChange={handleChange} required />
45            </label>
46          </div>
47          <div>
48            <label>
49              Temperature Control:
50              <select name="temperature" value={formState.temperature} onChange={handleChange}>
51                <option value={0}>No Control</option>
52                <option value={1}>Has Control</option>
53              </select>
54            </label>
55          </div>
56          <div>
57            <label>
58              Amperes Control:
59              <select name="amperes" value={formState.amperes} onChange={handleChange}>
60                <option value={0}>No Control</option>
61                <option value={1}>Has Control</option>
62              </select>
63            </label>
64          </div>
65          <div>
66            <label>
67              Decibels Control:
68              <select name="decibels" value={formState.decibels} onChange={handleChange}>
69                <option value={0}>No Control</option>
70                <option value={1}>Has Control</option>
71              </select>
72            </label>
73          </div>
74          <div>
75            <label>
76              Volume Control:
77              <select name="volume_percentage" value={formState.volume_percentage} onChange={handleChange}>
78                <option value={0}>No Control</option>
79                <option value={1}>Has Control</option>
80              </select>
81            </label>
82          </div>
83          <div>
84            <label>
85              State (ON/OFF):
86              <select name="Estado_dispositivo" value={formState.Estado_dispositivo} onChange={handleChange}>
87                <option value={0}>OFF</option>
88                <option value={1}>ON</option>
89              </select>
90            </label>
91          </div>
92          <button type="submit">Add Device</button>
93          <button type="button" onClick={onClose}>Cancel</button>
94        </form>
95      </div>
96    </div>
97  );
98 };
99
100 export default AddDispositivoModal;
101

```

Figura 38- Componente do EditPreferenciasModal

Apêndice F- Configuração dos CRUDS

```
1 const express = require('express');
2 const pool = require('pg');
3 const router = express.Router();
4
5 // GET all ambientes
6 router.get('/', (req, res) => {
7   pool.query('SELECT * FROM ambientes', (error, results) => {
8     if (error) {
9       console.error('Error fetching ambientes:', error);
10      res.status(500).send('Internal server error: ', error.message);
11      return;
12    }
13    res.send(results);
14  });
15});
16
17 // CREATE
18 router.post('/', (req, res) => {
19   const novoAmbiente = req.body;
20   pool.query('INSERT INTO ambientes SET ?', novoAmbiente, (error, results) => {
21     if (error) {
22       console.error('Error creating ambiente:', error);
23       res.status(500).send('Internal server error: ');
24       return;
25     }
26     res.send({ id: results.insertId, ...novoAmbiente });
27   });
28});
29
30 // UPDATE
31 router.put('/:id', (req, res) => {
32   const { id } = req.params;
33   const ambienteAtualizado = req.body;
34   pool.query('UPDATE ambientes SET ? WHERE id = ?', [ambienteAtualizado, id], (error, results) => {
35     if (error) {
36       console.error('Error updating ambiente:', error);
37       res.status(500).send('Internal server error: ');
38       return;
39     }
40     res.send(results);
41   });
42});
43
44 // DELETE ambiente by ID and associated records
45 router.delete('/:id', (req, res) => {
46   const { id } = req.params;
47
48   // Step 1: Delete checks associated with the ambiente
49   pool.query('DELETE FROM checks WHERE id_ambiente = ?', [id], (error) => {
50     if (error) {
51       return res.status(500).send('Internal server error: ');
52     }
53   });
54
55   // Step 2: Delete rules associated with the ambiente
56   pool.query('DELETE FROM rules WHERE id_ambiente = ?', [id], (error) => {
57     if (error) {
58       return res.status(500).send('Internal server error: ');
59     }
60   });
61
62   // Step 3: Delete the ambiente after deleting associated rules and checks
63   pool.query('DELETE FROM ambientes WHERE id_ambiente = ?', [id], (error, results) => {
64     if (error) {
65       return res.status(500).send('Internal server error: ');
66     }
67     res.status(200).send('Ambiente and associated records deleted successfully');
68   });
69 });
70
71 // Device temp by ambiente id
72 router.get('/:id', (req, res) => {
73   const { id } = req.params;
74   const query = `
75     SELECT
76       a.id,
77       d.dispositivo,
78       d.tipo_dispositivo,
79       d.temperatura,
80       d.umidade,
81       d.dB,
82       d.volume_percentagem,
83       a.valor_preferencia,
84       a.brightness,
85       a.relative_humidity,
86       a.sound,
87     FROM
88       ambientes a
89     JOIN
90       dispositivos d ON d.id_ambiente = a.id_ambiente
91     JOIN
92       checks c ON c.id_ambiente = a.id_ambiente
93     JOIN
94       preferencias p ON c.id_usuario = p.id_usuario
95     JOIN
96       usuarios u ON p.id_usuario = u.id_usuario
97     JOIN
98       roles r ON u.role_id = r.role_id
99     WHERE
100       c.data_criacao BETWEEN ? AND ? AND a.id_ambiente = ?
101     GROUP BY
102       a.id, d.dispositivo;
103   `;
104   console.log('Executing query with ID:', id);
105
106   pool.query(query, [id], (error, results) => {
107     if (error) {
108       console.error('Error fetching devices by ambiente ID:', error);
109       res.status(500).send('Internal server error: ');
110       return;
111     }
112
113     // Log the results to the console
114     console.log('Calculated values:', results);
115     res.status(200).send(results);
116   });
117});
118
119 module.exports = router;
```

Figura 39- Configuração do CRUD dos Ambientes

```

1 const express = require('express');
2 const pool = require('./db');
3 const router = express.Router();
4
5
6
7 //check-in
8 router.post('/checkin', (req, res) => {
9   const { ID_usuario, ID_ambiente } = req.body;
10   const checkInTime = new Date();
11
12   // fetch the user's role and check-in status
13   const userQuery = 'SELECT role_id, is_checked_in FROM usuarios WHERE ID_usuario = ?';
14   pool.query(userQuery, [ID_usuario], (err, userResult) => {
15     if (err) {
16       console.error('Error fetching user info:', err);
17       return res.status(500).send('Internal Server Error');
18     }
19
20     const { role_id: userRoleId, is_checked_in: isCheckedIn } = userResult[0];
21
22     if (isCheckedIn) {
23       return res.status(403).send('User is already checked into another ambiente');
24     }
25
26     // Check if the user's role is allowed in the specified ambiente
27     const allowedRoleQuery = 'SELECT * FROM roles_do_ambiente WHERE ID_ambiente = ? AND ID_role = ?';
28     pool.query(allowedRoleQuery, [ID_ambiente, userRoleId], (err, allowedRoleResult) => {
29       if (err) {
30         console.error('Error fetching allowed roles:', err);
31         return res.status(500).send('Internal Server Error');
32       }
33
34       if (allowedRoleResult.length === 0) {
35         return res.status(403).send('User role not allowed in this ambiente');
36       }
37
38       // Proceed with the check-in
39       const checkInQuery = 'INSERT INTO checkins (ID_usuario, ID_ambiente, check_in_time) VALUES (?, ?, ?)';
40       pool.query(checkInQuery, [ID_usuario, ID_ambiente, checkInTime], (error, results) => {
41         if (error) {
42           console.error('Error checking in user:', error);
43           return res.status(500).send('Internal Server Error');
44         }
45
46         // Update user's check-in status
47         const updateUserStatusQuery = 'UPDATE usuarios SET is_checked_in = TRUE WHERE ID_usuario = ?';
48         pool.query(updateUserStatusQuery, [ID_usuario], (updateError) => {
49           if (updateError) {
50             console.error('Error updating user status:', updateError);
51             return res.status(500).send('Internal Server Error');
52           }
53
54           res.status(201).send({ id: results.insertId, ID_usuario, ID_ambiente, checkInTime });
55         });
56       });
57     });
58   });
59 });
60
61 // Check out a user
62 router.post('/checkout', (req, res) => {
63   const { ID_usuario, ID_ambiente } = req.body;
64   const checkOutTime = new Date();
65
66   // Update check-out time for the latest check-in record of the user
67   const checkoutQuery = 'UPDATE checkins SET check_out_time = ? WHERE ID_usuario = ? AND ID_ambiente = ? AND check_out_time IS NULL';
68   pool.query(checkoutQuery, [checkOutTime, ID_usuario, ID_ambiente], (err, results) => {
69     if (err) {
70       console.error('Error checking out user:', err);
71       return res.status(500).send('Internal Server Error');
72     }
73
74     if (results.affectedRows === 0) {
75       return res.status(404).send('No active check-in found for the user in the specified ambiente');
76     }
77
78     // Update user's check-in status
79     const updateUserStatusQuery = 'UPDATE usuarios SET is_checked_in = FALSE WHERE ID_usuario = ?';
80     pool.query(updateUserStatusQuery, [ID_usuario], (updateError) => {
81       if (updateError) {
82         console.error('Error updating user status:', updateError);
83         return res.status(500).send('Internal Server Error');
84       }
85
86       res.status(200).send({ ID_usuario, ID_ambiente, checkOutTime });
87     });
88   });
89 });
90
91 // Get all users checked into a specific environment
92 router.get('/users/:ID_ambiente', (req, res) => {
93   const { ID_ambiente } = req.params;
94
95   const query = 'SELECT usuarios.*, roles.role_name, roles.role_weight, checkins.check_in_time FROM usuarios JOIN checkins ON usuarios.ID_usuario = checkins.ID_usuario LEFT JOIN roles ON usuarios.role_id = roles.role_id WHERE checkins.ID_ambiente = ? AND checkins.check_out_time IS NULL';
96   pool.query(query, [ID_ambiente], (error, results) => {
97     if (error) {
98       console.error('Error fetching checked-in users:', error);
99       res.status(500).send('Internal Server Error');
100     }
101     return;
102   });
103   res.status(200).send(results);
104 });
105
106 module.exports = router;

```

Figura 40- Configuração do CRUD dos Chekins

```

1  const express = require('express');
2
3  const pool = require('../db'); // Assuming db.js is in the same directory
4
5  const router = express.Router();
6
7
8  // CREATE a new dispositivo
9  router.post('/', (req, res) => {
10     const newDispositivo = req.body; // Ensure all fields are present in req.body
11
12     // Insert the new dispositivo into the database
13     pool.query('INSERT INTO dispositivos SET ?', newDispositivo, (error, results) => {
14         if (error) {
15             console.error('Error adding dispositivo to ambiente:', error);
16             res.status(500).send('Internal Server Error');
17             return;
18         }
19         const dispositivoId = results.insertId;
20         res.status(201).json({ ID_dispositivo: dispositivoId });
21     });
22 });
23
24
25
26
27 // READ - Get all dispositivos
28 router.get('/', (req, res) => {
29     pool.query('SELECT * FROM Dispositivos', (error, results) => {
30         if (error) {
31             console.error('Error fetching dispositivos:', error);
32             res.status(500).send('Internal Server Error' + error.message);
33             return;
34         }
35         res.send(results);
36     });
37 });
38
39 // UPDATE - Update a dispositivo by ID
40 router.put('/:id', (req, res) => {
41     const { id } = req.params;
42     const dispositivoAtualizado = req.body;
43
44     const {
45         ID_ambiente,
46         Tipo_dispositivo,
47         Estado_dispositivo,
48         temperatura,
49         amperes,
50         decibels,
51         volume_percentage,
52     } = dispositivoAtualizado;
53
54     const query = `
55     UPDATE Dispositivos
56     SET
57         ID_ambiente = ?,
58         Tipo_dispositivo = ?,
59         Estado_dispositivo = ?,
60         temperatura = ?,
61         amperes = ?,
62         decibels = ?,
63         Volume_percentage = ?
64     WHERE ID_dispositivo = ?
65     `;
66
67     const values = [
68         ID_ambiente,
69         Tipo_dispositivo,
70         Estado_dispositivo,
71         temperatura,
72         amperes,
73         decibels,
74         volume_percentage,
75         id
76     ];
77
78     pool.query(query, values, (error, results) => {
79         if (error) {
80             console.error('Error updating dispositivo:', error);
81             res.status(500).send('Internal Server Error' + error.message);
82             return;
83         }
84         res.send(results);
85     });
86 });
87
88 // DELETE - Delete a dispositivo by ID
89 router.delete('/:id', (req, res) => {
90     const { id } = req.params;
91     pool.query('DELETE FROM Dispositivos WHERE ID_dispositivo = ?', id, (error, results) => {
92         if (error) {
93             console.error('Error deleting dispositivo:', error);
94             res.status(500).send('Internal Server Error' + error.message);
95             return;
96         }
97         res.send(results);
98     });
99 });
100
101 // FETCH dispositivos by ID_ambiente
102 router.get('/ambientes/:ID_ambiente', (req, res) => {
103     const { ID_ambiente } = req.params;
104     pool.query('SELECT * FROM dispositivos WHERE ID_ambiente = ?', [ID_ambiente], (error, results) => {
105         if (error) {
106             console.error('Error fetching dispositivos:', error);
107             res.status(500).send('Internal Server Error');
108             return;
109         }
110         res.status(200).json(results);
111     });
112 });
113
114
115
116
117
118
119
120
121 module.exports = router;
122

```

Figura 41- Configuração do CRUD dos Dispositivos

```

1  const express = require('express');
2  const pool = require('./db');
3  const router = express.Router();
4
5  // READ - Get all preferences
6  router.get('/', (req, res) => {
7    pool.query('SELECT * FROM preferencias', (error, results) => {
8      if (error) {
9        console.error('Error fetching preferences:', error);
10       res.status(500).send('Internal Server Error'+error.message);
11       return;
12     }
13     res.send(results);
14   });
15 });
16
17 // CREATE a new preference
18 router.post('/', (req, res) => {
19   const newPreference = req.body;
20   console.log('Received preference data:', newPreference);
21
22   pool.query('INSERT INTO preferencias SET ?', newPreference, (error, results) => {
23     if (error) {
24       console.error('Error creating preference:', error);
25       res.status(500).send('Internal Server Error');
26       return;
27     }
28     res.status(201).json({ insertId: results.insertId });
29   });
30 });
31
32 // UPDATE
33 router.put('/:id', (req, res) => {
34   const { id } = req.params;
35   const updatePreferences = req.body;
36   pool.query('UPDATE preferencias SET ? WHERE ID_preferencia = ?', [updatePreferences, id], (error, results) => {
37     if (error) {
38       console.error('Error updating preference:', error);
39       res.status(500).send('Internal Server Error');
40       return;
41     }
42     res.send(results);
43   });
44 });
45
46 // READ - Get preferences by user ID
47 router.get('/:userId', (req, res) => {
48   const userId = req.params.userId;
49   pool.query('SELECT * FROM preferencias WHERE ID_usuario = ?', userId, (error, results) => {
50     if (error) {
51       console.error('Error fetching preferences:', error);
52       res.status(500).send('Internal Server Error');
53       return;
54     }
55     res.send(results);
56   });
57 });
58
59 module.exports = router;
60
61

```

Figura 42- Configuração do CRUD das Preferências


```

1
2 const express = require('express');
3 const pool = require('./db');
4 const router = express.Router();
5
6 // Read all registros
7 router.get('/', (req, res) => {
8     pool.query('SELECT * FROM registros', (error, results) => {
9         if (error) {
10             console.error('Error fetching registros:', error);
11             res.status(500).send('Internal Server Error'+error.message);
12             return;
13         }
14         res.send(results);
15     });
16 });
17
18 // Read a single registro by ID
19 router.get('/:id', async (req, res) => {
20     const { id } = req.params;
21     try {
22         const [rows] = await pool.query('SELECT * FROM registros WHERE ID_registro = ?', [id]);
23         if (rows.length === 0) {
24             return res.status(404).json({ error: 'Registro not found' });
25         }
26         res.status(200).json(rows[0]);
27     } catch (error) {
28         console.error('Error fetching registro:', error);
29         res.status(500).json({ error: 'Internal Server Error' });
30     }
31 });
32
33
34 // Delete a registro by ID
35 router.delete('/:id', async (req, res) => {
36     const { id } = req.params;
37     try {
38         const [result] = await pool.query('DELETE FROM registros WHERE ID_registro = ?', [id]);
39         if (result.affectedRows === 0) {
40             return res.status(404).json({ error: 'Registro not found' });
41         }
42         res.status(200).json({ message: 'Registro deleted successfully' });
43     } catch (error) {
44         console.error('Error deleting registro:', error);
45         res.status(500).json({ error: 'Internal Server Error' });
46     }
47 });
48
49 module.exports = router;
50

```

Figura 43- Configuração do CRUD dos Registros

```

1  const express = require('express');
2  const pool = require('./db');
3  const router = express.Router();
4
5  // CREATE a new role
6  router.post('/', (req, res) => {
7    const newRole = req.body;
8    pool.query('INSERT INTO roles SET ?', newRole, (error, results) => {
9      if (error) {
10        console.error('Error creating role:', error);
11        res.status(500).send('Internal Server Error');
12        return;
13      }
14      res.status(201).send({ id: results.insertId });
15    });
16  });
17
18  // READ all roles
19  router.get('/', (req, res) => {
20    pool.query('SELECT * FROM roles', (error, results) => {
21      if (error) {
22        console.error('Error fetching roles:', error);
23        res.status(500).send('Internal Server Error');
24        return;
25      }
26      res.send(results);
27    });
28  });
29
30  // READ a single role by ID
31  router.get('/:id', (req, res) => {
32    const { id } = req.params;
33    pool.query('SELECT * FROM roles WHERE role_id = ?', [id], (error, results) => {
34      if (error) {
35        console.error('Error fetching role:', error);
36        res.status(500).send('Internal Server Error');
37        return;
38      }
39      if (results.length === 0) {
40        res.status(404).send('Role not found');
41      } else {
42        res.send(results[0]);
43      }
44    });
45  });
46
47  // Update a role by ID
48  router.put('/:id', (req, res) => {
49    const { id } = req.params;
50    const updatedRole = req.body;
51    pool.query('UPDATE roles SET ? WHERE role_id = ?', [updatedRole, id], (error, results) => {
52      if (error) {
53        console.error('Error updating role:', error);
54        res.status(500).send('Internal Server Error');
55        return;
56      }
57      res.status(200).send('Role updated successfully');
58    });
59  });
60
61  // DELETE a role by ID
62  router.delete('/:id', (req, res) => {
63    const { id } = req.params;
64    pool.query('DELETE FROM roles WHERE role_id = ?', [id], (error, results) => {
65      if (error) {
66        console.error('Error deleting role:', error);
67        res.status(500).send('Internal Server Error');
68        return;
69      }
70      res.send('Role deleted successfully');
71    });
72  });
73
74
75
76
77  module.exports = router;
78

```

Figura 44- Configuração do CRUD dos Roles

```

1  const express = require('express');
2  const pool = require('./db'); // Adjust the path as needed
3
4  const router = express.Router();
5  // CREATE
6  router.post('/add', (req, res) => {
7      const { ID_role, ID_ambiente } = req.body;
8
9      // Check if the role already exists in the environment
10     const checkQuery = 'SELECT * FROM roles_do_ambiente WHERE ID_role = ? AND ID_ambiente = ?';
11     pool.query(checkQuery, [ID_role, ID_ambiente], (checkErr, checkResults) => {
12         if (checkErr) {
13             console.error('Error checking existing role:', checkErr);
14             return res.status(500).send('Internal Server Error');
15         }
16         if (checkResults.length > 0) {
17             // Role already exists in the environment
18             return res.status(400).send('Role already exists in the environment');
19         }
20
21         // Role doesn't exist, proceed with insertion
22         const insertQuery = 'INSERT INTO roles_do_ambiente (ID_role, ID_ambiente) VALUES (?, ?)';
23         pool.query(insertQuery, [ID_role, ID_ambiente], (insertErr, insertResults) => {
24             if (insertErr) {
25                 console.error('Error adding role to environment:', insertErr);
26                 return res.status(500).send('Internal Server Error');
27             }
28             res.status(201).send({ ID_role_do_ambiente: insertResults.insertId });
29         });
30     });
31 });
32
33 // FETCH roles_do_ambiente by ID_ambiente and include role_name
34 router.get('/:ID_ambiente', (req, res) => {
35     const { ID_ambiente } = req.params;
36
37     pool.query('
38         SELECT rda.*, r.role_name
39         FROM roles_do_ambiente AS rda
40         JOIN roles AS r ON rda.ID_role = r.role_id
41         WHERE rda.ID_ambiente = ?
42     ', [ID_ambiente], (error, results) => {
43         if (error) {
44             console.error('Error fetching roles_do_ambiente:', error);
45             res.status(500).send('Internal Server Error');
46             return;
47         }
48         res.status(200).json(results);
49     });
50 });
51
52 // UPDATE
53 router.put('/:id', (req, res) => {
54     const { id } = req.params;
55     const updatedRoleDoAmbiente = req.body;
56     pool.query('UPDATE roles_do_ambiente SET ? WHERE ID_role_do_ambiente = ?', [updatedRoleDoAmbiente, id], (error, results) => {
57         if (error) {
58             console.error('Error updating role_do_ambiente:', error);
59             res.status(500).send('Internal Server Error');
60             return;
61         }
62         res.send(results);
63     });
64 });
65
66 // DELETE
67 router.delete('/:id', (req, res) => {
68     const { id } = req.params;
69     pool.query('DELETE FROM roles_do_ambiente WHERE ID_role_do_ambiente = ?', id, (error, results) => {
70         if (error) {
71             console.error('Error deleting role_do_ambiente:', error);
72             res.status(500).send('Internal Server Error');
73             return;
74         }
75         res.send(results);
76     });
77 });
78
79 module.exports = router;
80
81
82

```

Figura 45- Configuração do CRUD dos Roles do Ambiente

Apêndice F- Configuração das Páginas

```
1
2 /* Close Button */
3 .close {
4   color: #aaa;
5   float: right;
6   font-size: 28px;
7   font-weight: bold;
8 }
9
10 .close:hover,
11 .close:focus {
12   color: black;
13   text-decoration: none;
14   cursor: pointer;
15 }
16
17 /* Form Styles */
18 label {
19   display: block;
20   margin-bottom: 10px;
21 }
22
23 input[type='text'],
24 input[type='number'],
25 select {
26   width: 20%;
27   padding: 8px;
28   border: 1px solid #ccc;
29   border-radius: 4px;
30   box-sizing: border-box;
31   margin-top: 5px;
32 }
33
34 /* Button Styles */
35 button {
36   background-color: #007bff;
37   color: white;
38   padding: 10px 20px;
39   border: none;
40   border-radius: 4px;
41   cursor: pointer;
42   margin-top: 10px;
43 }
44
45 button:hover {
46   background-color: #0056b3;
47 }
48
49 .device-container {
50   display: flex;
51   display: inline-block;
52 }
53
54 .device-square {
55   width: 240px;
56   height: 200px;
57   background-color: #ccc;
58   display: inline-block;
59   margin-right: 1px;
60 }
```

Figura 47- Configuração da página de EnterPageCss



```
1 import React from 'react';
2 import AmbientesList from './AmbientesList';
3
4 const AmbientesPage = () => {
5   return (
6     <div>
7       <h1>    </h1>
8       <AmbientesList />
9     </div>
10  );
11 };
12 export default AmbientesPage;
13
```

Figura 48- Configuração da página dos Ambientes

[illegible]

Figura 49- Configuração da página do AmbientesList

```

1  /* Styles for the ambientes list container */
2  .ambientes-list {
3      padding: 20px;
4      background-color: #e0e0e0;
5      border-radius: 8px;
6      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
7  }
8
9  /* Styles for the list items */
10 .ambiente-item {
11     margin-bottom: 10px;
12     padding: 10px;
13     background-color: #fff;
14     border-radius: 4px;
15     box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
16     display: flex;
17     align-items: center;
18     justify-content: space-between;
19 }
20
21 /* Styles for the ambiente info */
22 .ambiente-info {
23     margin-right: 10px;
24     width: 100px;
25 }
26
27 /* Styles for the button group */
28 .button-group {
29     display: flex;
30     gap: 5px;
31     justify-content: flex-end;
32 }
33
34 /* Styles for the buttons */
35 .button {
36     padding: 8px 16px;
37     border: none;
38     border-radius: 4px;
39     background-color: #007bff;
40     color: #fff;
41     cursor: pointer;
42 }
43
44 .button_delete {
45     padding: 8px 16px;
46     border: none;
47     border-radius: 4px;
48     background-color: #3f4852;
49     color: #fff;
50     cursor: pointer;
51 }
52
53 .button:hover {
54     background-color: #0056b3;
55 }
56
57 .button_delete:hover {
58     background-color: #2c3e50;
59 }
60
61 /* Styles for the header */
62 .ambientes-list-header {
63     display: flex;
64     align-items: center;
65     justify-content: space-between;
66 }
67
68 .ambientes-list-header h2 {
69     font-size: 24px;
70     margin-bottom: 20px;
71     color: #333;
72 }
73
74 /* Styles for the add ambiente button */
75 .button_add_ambiente {
76     padding: 8px 16px;
77     border: none;
78     border-radius: 4px;
79     background-color: #007bff;
80     color: #fff;
81     cursor: pointer;
82 }
83
84 .button_add_ambiente:hover {
85     background-color: #0056b3;
86 }
87
88
89
90
91

```

Figura 50- Configuração da página do AmbientesListCss





```
1  import React from 'react';
2  import RolesList from './RolesList';
3
4  const RolesPage = () => {
5    return (
6      <div>
7        <h1>      </h1>
8        <RolesList />
9      </div>
10   );
11 };
12 export default RolesPage;
13
```

Figura 52- Configuração da página dos Roles

```

1 import React, { useState, useEffect } from 'react';
2 import { fetchRoles, createRole, deleteRole, updateRole } from '../api/roles.js';
3 import RoleForm from '../components/roles/RoleForm.js';
4 import EditRoleModal from '../components/roles/EditRoleModal.js';
5 import './Roles.css';
6
7 const RolesList = () => {
8   const [roles, setRoles] = useState([]);
9   const [loading, setLoading] = useState(true);
10  const [error, setError] = useState(null);
11  const [showModal, setShowModal] = useState(false);
12  const [editingRole, setEditingRole] = useState(null);
13  const [confirmDelete, setConfirmDelete] = useState(false);
14  const [roleToDelete, setRoleToDelete] = useState(null);
15
16  const fetchData = async () => {
17    try {
18      const rolesData = await fetchRoles();
19      setRoles(rolesData);
20    } catch (error) {
21      setError(error.message);
22    } finally {
23      setLoading(false);
24    }
25  };
26
27  useEffect(() => {
28    fetchData();
29  }, []);
30
31  // Effect to refetch roles data whenever roles state changes
32  useEffect(() => {
33    fetchData();
34  }, [roles]);
35
36  const handleAddRole = async (roleData) => {
37    try {
38      const newRole = await createRole(roleData);
39      setRoles([...roles, newRole]);
40      setShowModal(false);
41    } catch (error) {
42      console.error('Error creating role:', error);
43    }
44  };
45
46  const handleEditRole = async (editedRole) => {
47    try {
48      await updateRole(editingRole.role_id, editedRole);
49      setRoles(roles.map(role => (role.role_id === editingRole.role_id ? editedRole : role)));
50      setEditingRole(null);
51      setShowModal(false);
52    } catch (error) {
53      console.error('Error updating role:', error);
54    }
55  };
56
57  const handleDeleteRole = async (roleId) => {
58    setRoleToDelete(roleId);
59    setConfirmDelete(true);
60  };
61
62  const confirmDeleteRole = async () => {
63    try {
64      await deleteRole(roleToDelete);
65      setRoles(roles.filter(role => role.role_id !== roleToDelete));
66      setConfirmDelete(false);
67      window.location.reload(); // Refresh the page after successful deletion
68    } catch (error) {
69      console.error('Error deleting role:', error);
70    }
71  };
72
73  const cancelDelete = () => {
74    setRoleToDelete(null);
75    setConfirmDelete(false);
76  };
77
78  if (loading) {
79    return <p>Loading roles...</p>;
80  }
81
82  if (error) {
83    return <p>Error fetching roles: {error}</p>;
84  }
85
86  return (
87    <div className="roles-list">
88      <div className="roles_list_header">
89        <h2>Roles List</h2>
90        <button className="button add_role" onClick={() => setShowModal(true)}>Add Role</button>
91      </div>
92      <ul>
93        {roles.map(role => (
94          <li key={role.role_id} className="role-item">
95            <div className="role-info">
96              {role.role_name} - {role.role_weight}
97            </div>
98            <div>
99              <button className="button edit" onClick={() => setEditingRole(role); setShowModal(true)}>Edit</button>
100              <button className="button delete" onClick={() => handleDeleteRole(role.role_id)}>Remove</button>
101            </div>
102          </li>
103        ))}
104      </ul>
105      <div>
106        <showModal && {
107          editingRole ?
108            <FullRoleModal>
109              role={editingRole}
110              onClose={() => setEditingRole(null); setShowModal(false); }
111              onSave={handleEditRole}
112            </FullRoleModal>
113            :
114            <RoleForm>
115              onClose={() => setShowModal(false)}
116              onSave={handleAddRole}
117            </RoleForm>
118          </showModal>
119        )}
120        <div>
121          <confirmDelete && {
122            roleToDelete ?
123              <DeleteConfirmationModal>
124                role={roleToDelete}
125                onClose={() => setRoleToDelete(null); setConfirmDelete(false); }
126                onConfirm={confirmDeleteRole}
127              </DeleteConfirmationModal>
128              :
129              </confirmDelete>
130            </div>
131          </div>
132        </div>
133      </div>
134    </div>
135  );
136
137  const DeleteConfirmationModal = ({ onClose, onConfirm, role }) => {
138    return (
139      <div className="modal">
140        <div className="modal-content">
141          <h3>Confirm Delete</h3>
142          <p>Are you sure you want to delete role: {role.role_name}</p>
143          <div className="modal-actions">
144            <button onClick={() => { onConfirm(); onClose(); }}>Confirm</button>
145            <button onClick={onClose}>Cancel</button>
146          </div>
147        </div>
148      </div>
149    );
150  };
151
152  export default RolesList;
153
154

```

Figura 53- Configuração da página do RolesList

```

1  /* Container for the rules list */
2  .rules-list {
3    padding: 20px;
4    background-color: #f0f0f0;
5    border-radius: 8px;
6    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
7  }
8
9  /* Header for the rules list */
10 .h2 {
11   font-size: 24px; /* Adjusted the font size for consistency */
12   margin-bottom: 20px;
13   color: #333;
14 }
15
16 /* Button to add a new rule */
17 .button-add-rule {
18   padding: 8px 16px;
19   border: none;
20   border-radius: 4px;
21   background-color: #007bff;
22   color: #fff;
23   cursor: pointer;
24   margin-bottom: 20px;
25 }
26
27 .button-add-rule:hover {
28   background-color: #0056b3;
29 }
30
31 /* Styles for the rules list items */
32 .rules-list ul {
33   list-style-type: none;
34   margin: 0;
35   padding: 0;
36 }
37
38 .rules-list li {
39   margin-bottom: 10px;
40   padding: 10px;
41   background-color: #fff;
42   border-radius: 4px;
43   box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
44   display: flex;
45   justify-content: space-between;
46   align-items: center;
47 }
48
49 /* Modal styles */
50 .modal {
51   display: flex;
52   justify-content: center;
53   align-items: center;
54   position: fixed;
55   width: 100%;
56   height: 100%;
57   top: 0;
58   left: 0;
59   width: 100%;
60   height: 100%;
61   overflow: auto;
62   background-color: rgba(0, 0, 0, 0.4);
63 }
64
65 .modal-content {
66   background-color: #f0f0f0;
67   margin: auto;
68   padding: 20px;
69   border: 1px solid #ddd;
70   width: 80%;
71   max-width: 500px;
72   border-radius: 8px;
73 }
74
75 .close {
76   color: #aaa;
77   float: right;
78   font-size: 24px;
79   font-weight: bold;
80 }
81
82 .close:hover,
83 .close:focus {
84   color: black;
85   text-decoration: none;
86   cursor: pointer;
87 }
88
89 .modal h2 {
90   margin-top: 0;
91 }
92
93 .button-edit {
94   position: absolute;
95   right: 10px;
96   padding: 8px 16px;
97   border: none;
98   border-radius: 4px;
99   background-color: #007bff;
100  color: #fff;
101  cursor: pointer;
102 }
103
104 .button-delete {
105   padding: 8px 16px;
106   border: none;
107   border-radius: 4px;
108   background-color: #dc3545;
109   color: #fff;
110   cursor: pointer;
111 }
112
113 .button-edit:hover {
114   background-color: #0056b3;
115 }
116
117 .button-delete:hover {
118   background-color: #c0392b;
119 }
120
121
122 .rule-item {
123   margin-bottom: 10px;
124   padding: 10px;
125   background-color: #fff;
126   border-radius: 4px;
127   box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
128   display: flex;
129   align-items: center;
130   justify-content: flex-start;
131 }
132
133 .h2 {
134   font-size: 24px; /* Adjusted the font size for consistency */
135   margin-bottom: 20px;
136   color: #333;
137 }
138
139 .rules-list header {
140   display: flex;
141   align-items: center;
142   justify-content: space-between;
143 }

```

Figura 54- Configuração da página do RolesCss



```
1  import React from 'react';
2  import UserList from './UserList';
3
4  const UsersPage = () => {
5    return (
6      <div>
7        <h1>    </h1>
8        <UserList />
9      </div>
10   );
11 };
12
13 export default UsersPage;
14
```

Figura 55- Configuração da página do UserList

```

1 import React, { useState, useEffect } from 'react';
2 import { fetchUsers, deleteUser, createUser } from '../api/users';
3 import EditUserModal from '../components/users/EditUserModal';
4 import AddUserModal from '../components/users/AddUserModal';
5 import './UserList.css';
6
7 const UserList = () => {
8   const [users, setUsers] = useState([]);
9   const [isLoading, setIsLoading] = useState(true);
10   const [error, setError] = useState(null);
11   const [editingUser, setEditingUser] = useState(null);
12   const [addingUser, setAddingUser] = useState(false);
13   const [confirmDelete, setConfirmDelete] = useState(false);
14   const [userToDelete, setUserToDelete] = useState(null);
15
16   const loadUsers = async () => {
17     try {
18       const data = await fetchUsers();
19       setUsers(data);
20     } catch (error) {
21       setError(error.message);
22     } finally {
23       setIsLoading(false);
24     }
25   };
26
27   useEffect(() => {
28     loadUsers();
29   }, []);
30
31   const handleDelete = async (userId) => {
32     setUserToDelete(userId);
33     setConfirmDelete(true);
34   };
35
36   const confirmDeleteUser = async () => {
37     try {
38       await deleteUser(userToDelete);
39       setUsers(users.filter(user => user.id !== userToDelete.id));
40       setConfirmDelete(false);
41       setTimeout(() => { // Refresh the page after successful deletion
42         loadUsers();
43       }, 1000);
44     } catch (error) {
45       console.error('Error deleting user:', error);
46     }
47   };
48
49   const cancelDelete = () => {
50     setUserToDelete(null);
51     setConfirmDelete(false);
52   };
53
54   const handleEdit = (user) => {
55     setEditingUser(user);
56   };
57
58   const handleCloseModal = () => {
59     setEditingUser(null);
60     setAddingUser(false);
61   };
62
63   const handleSaveUser = (updateUser) => {
64     setUser(editingUser ? updateUser : { id: editingUser.id, username: editingUser.username, role: editingUser.role });
65     setEditingUser(null);
66   };
67
68   const handleAddUser = () => {
69     setAddingUser(true);
70   };
71
72   const handleSaveUser = async (newUser) => {
73     try {
74       const createdUser = await createUser(newUser);
75       setUsers([...users, createdUser]);
76       setAddingUser(false);
77     } catch (error) {
78       console.error('Error adding user:', error);
79     }
80   };
81
82   // Loading users...
83   return (
84     <div>
85       <div className="user-list">
86         <div className="user-list-header">
87           <div>User List</div>
88           <button className="button_add_user" onClick={handleAddUser}>Add User</button>
89         </div>
90         <div>
91           {users.map(user => (
92             <div key={user.id} className="user-item">
93               <div className="user-info">
94                 <div>User Name: {user.username}</div>
95                 <div>Role: {user.role}</div>
96               </div>
97               <div>
98                 <button className="button_edit" onClick={() => handleEdit(user)}>Edit</button>
99                 <button className="button_delete" onClick={() => handleDelete(user.id)}>Delete</button>
100               </div>
101             </div>
102           ))}
103         </div>
104       </div>
105       <div>
106         <div>
107           <div>
108             <div>
109               <div>
110                 <div>
111                   <div>
112                     <div>
113                       <div>
114                         <div>
115                           <div>
116                             <div>
117                               <div>
118                                 <div>
119                                   <div>
120                                     <div>
121                                       <div>
122                                         <div>
123                                           <div>
124                                             <div>
125                                             </div>
126                                           </div>
127                                         </div>
128                                       </div>
129                                     </div>
130                                   </div>
131                                 </div>
132                               </div>
133                             </div>
134                           </div>
135                         </div>
136                       </div>
137                     </div>
138                   </div>
139                 </div>
140               </div>
141             </div>
142           </div>
143         </div>
144       </div>
145     </div>
146   );
147
148   const DeleteConfirmationModal = ({ onClose, onConfirm, user }) => {
149     return (
150       <div className="modal">
151         <div className="modal-content">
152           <div>
153             <div>
154               <div>
155                 <div>
156                   <div>
157                     <div>
158                       <div>
159                         <div>
160                           <div>
161                             <div>
162                               <div>
163                                 <div>
164                                   <div>
165                                     <div>
166                                       <div>
167                                         <div>
168                                           <div>
169                                             <div>
170                                             </div>
171                                           </div>
172                                         </div>
173                                       </div>
174                                     </div>
175                                   </div>
176                                 </div>
177                               </div>
178                             </div>
179                           </div>
180                         </div>
181                       </div>
182                     </div>
183                   </div>
184                 </div>
185               </div>
186             </div>
187           </div>
188         </div>
189       </div>
190     );
191   };
192
193   export default UserList;
194 }

```

Figura 56- Configuração da página do UserList

```

1  /* Styles for the user list container */
2  .user-list {
3      padding: 20px;
4      background-color: #e0e0e0; /* Changed background color for testing */
5      border-radius: 8px;
6      box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
7  }
8
9  /* Styles for the list items */
10 .user-item {
11     margin-bottom: 10px;
12     padding: 10px;
13     background-color: #fff;
14     border-radius: 4px;
15     box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
16     display: flex;
17     align-items: center;
18     justify-content: flex-start;
19 }
20
21 /* Styles for the user info */
22 .user-info {
23     margin-right: 10px;
24     width: 100px;
25 }
26 .static-user-info {
27     position: relative;
28     left: 40px;
29 }
30 /* Styles for the button group */
31 .button-group {
32     display: flex;
33     gap: 5px;
34     justify-content: flex-end
35 }; /* Align to the left */
36 }
37
38 /* Styles for the buttons */
39 .button {
40     padding: 8px 16px;
41     border: none;
42     border-radius: 4px;
43     background-color: #007bff;
44     color: #fff;
45     cursor: pointer;
46 }
47
48 .button_delete {
49     padding: 8px 16px;
50     border: none;
51     border-radius: 4px;
52     background-color: #3f4852;
53     color: #fff;
54     cursor: pointer;
55 }
56 .button_role_weight {
57     padding: 8px 16px;
58     border: none;
59     border-radius: 4px;
60     background-color: #4b7fba;
61     color: #fff;
62     cursor: pointer;
63 }
64
65 .button:hover {
66     background-color: #0056b3;
67 }
68 .user-list-header {
69     display: flex;
70     align-items: center;
71     right: 200px;
72 }
73
74 .user-list-header h2 {
75     position: relative;
76     left: 40px;
77 }
78
79 .button_add_user {
80     position: sticky;
81     padding: 8px 16px;
82     border: none;
83     border-radius: 4px;
84     background-color: #007bff;
85     color: #fff;
86     cursor: pointer;
87     left: 1060px;
88     top: 0;
89 }
90
91 .button_add_user:hover {
92     background-color: #0056b3;
93 }
94
95 /* Styles for the headers */
96 h2 {
97     font-size: 24px; /* Adjusted the font size for consistency */
98     margin-bottom: 20px;
99     color: #333;
100 }
101
102

```

Figura 57- Configuração da página do UserListCss

```

1 // src/pages/RegistrosList.js
2 import React, { useEffect, useState } from 'react';
3 import { fetchAllRegistros, deleteRegistroById } from '../../api/registros';
4 import './Registros.css';
5
6 const RegistrosList = () => {
7   const [registros, setRegistros] = useState([]);
8   const [loading, setLoading] = useState(true);
9   const [error, setError] = useState(null);
10  const [confirmDelete, setConfirmDelete] = useState(false);
11  const [registroToDelete, setRegistroToDelete] = useState(null);
12
13  useEffect(() => {
14    const loadRegistros = async () => {
15      try {
16        const data = await fetchAllRegistros();
17        setRegistros(data);
18      } catch (error) {
19        setError(error.message);
20      } finally {
21        setLoading(false);
22      }
23    };
24
25    loadRegistros();
26  }, []);
27
28  const handleDelete = async (registroId) => {
29    setRegistroToDelete(registroId);
30    setConfirmDelete(true);
31  };
32
33  const confirmDeleteRegistro = async () => {
34    try {
35      await deleteRegistroById(registroToDelete);
36      setRegistros(registros.filter((registro) => registro.ID_registro !== registroToDelete));
37      setConfirmDelete(false);
38    } catch (error) {
39      console.error('Error deleting registro:', error);
40    }
41  };
42
43  const cancelDelete = () => {
44    setRegistroToDelete(null);
45    setConfirmDelete(false);
46  };
47
48  if (loading) {
49    return <p>Loading Registros...</p>;
50  }
51
52  if (error) {
53    return <p>Error fetching Registros: {error}</p>;
54  }
55
56  return (
57    <div className="registros-list">
58      <div className="registros-list-header">
59        <h2>Registros List</h2>
60      </div>
61      <ul>
62        {registros.map((registro) => (
63          <li key={registro.ID_registro} className="registro-item">
64            <div className="registro-info">
65              Ambiente ID: {registro.ID_ambiente} - Calculated Temperature: {registro.calculated_temperature} C°
66              - Sound: {registro.sound} - Brightness: {registro.brightness} - Humidity: {registro.humidity} - Date: {registro.timestamp}
67            </div>
68            <div className="button-group">
69              <button className="button_delete_r" onClick={() => handleDelete(registro.ID_registro)}>
70                Delete
71              </button>
72            </div>
73          </li>
74        ))}
75      </ul>
76      {confirmDelete && (
77        <DeleteConfirmationModal
78          onClose={cancelDelete}
79          onConfirm={confirmDeleteRegistro}
80          registro={registros.find((registro) => registro.ID_registro === registroToDelete)}
81        />
82      )}
83    </div>
84  );
85 };
86
87 const DeleteConfirmationModal = ({ onClose, onConfirm, registro }) => {
88   return (
89     <div className="modal">
90       <div className="modal-content">
91         <h2>Confirm Delete</h2>
92         <p>Are you sure you want to delete the registro with ID: {registro.ID_registro}?</p>
93         <div className="modal-buttons">
94           <button onClick={onConfirm}>Confirm</button>
95           <button onClick={onClose}>Cancel</button>
96         </div>
97       </div>
98     </div>
99   );
100 };
101
102 export default RegistrosList;
103

```

Figura 58- Configuração da página do RegistrosList


```

1  /* src/pages/RegistrosList.css */
2  .registros-list {
3      padding: 20px;
4  }
5
6  .registros-list-header {
7      display: flex;
8      justify-content: space-between;
9      align-items: center;
10 }
11
12 .registro-item {
13     display: flex;
14     justify-content: space-between;
15     align-items: center;
16     padding: 10px;
17     border-bottom: 1px solid #ccc;
18 }
19
20 .registro-info {
21     flex: 1;
22 }
23
24 .button-group {
25     display: flex;
26     gap: 10px;
27 }
28
29 .button_delete_r {
30     background-color: rgb(15, 14, 14);
31     color: white;
32     border: none;
33     padding: 5px 10px;
34     cursor: pointer;
35 }
36
37 .button_delete:hover {
38     background-color: rgb(49, 47, 47);
39 }
40
41 .modal {
42     position: fixed;
43     top: 0;
44     left: 0;
45     width: 100%;
46     height: 100%;
47     background: rgba(0, 0, 0, 0.5);
48     display: flex;
49     justify-content: center;
50     align-items: center;
51 }
52
53 .modal-content {
54     background: white;
55     padding: 20px;
56     border-radius: 4px;
57 }
58
59 .modal-buttons {
60     display: flex;
61     justify-content: flex-end;
62     gap: 10px;
63 }
64
65 .modal-buttons button {
66     padding: 5px 10px;
67 }
68

```

Figura 59- Configuração da página do RegistosCss

```

1  import React from 'react';
2  import { BrowserRouter as Router, Route, Routes, Link } from 'react-router-dom';
3  import UsersPage from './pages/Users/UsersPage';
4  import RolesPage from './pages/Roles/RolesPage';
5  import AmbientesPage from './pages/Ambientes/AmbientesPage';
6  import EnterAmbientePage from './pages/Ambientes/EnterAmbientePage';
7  import RegistosList from './pages/Registos/RegistosList';
8  import './App.css';
9
10 function App() {
11   return (
12     <Router>
13       <div className="App">
14         <nav className="navbar">
15           <ul>
16             <li>
17               <Link to="/">Environments</Link>
18             </li>
19             <li>
20               <Link to="/users">Users</Link>
21             </li>
22             <li>
23               <Link to="/roles">Roles</Link>
24             </li>
25             <li>
26               <Link to="/registos">Records</Link>
27             </li>
28           </ul>
29         </nav>
30         <Routes>
31           <Route path="/roles" element={<RolesPage />} />
32           <Route path="/users" element={<UsersPage />} />
33           <Route path="/" element={<AmbientesPage />} />
34           <Route path="/ambientes/:ambienteId" element={<EnterAmbientePage />} />
35           <Route path="/registos" element={<RegistosList />} />
36         </Routes>
37       </div>
38     </Router>
39   );
40 }
41
42 export default App;
43
44

```

Figura 60- Configuração da página da App



```
1  const express = require('express');
2  const app = express();
3  const bodyParser = require('body-parser');
4  const pool = require('./cruds/db');
5  const cors = require('cors');
6
7  app.use(bodyParser.json());
8
9  const usersRoutes = require('./cruds/usuarios');
10 const ambientesRoutes = require('./cruds/ambientes');
11 const dispositivosRoutes = require('./cruds/dispositivos');
12 const checkinRoutes = require('./cruds/checkin');
13 const roleRoutes=require('./cruds/roles');
14 const rolesDoAmbienteRoutes = require('./cruds/rolesDoAmbiente');
15 const preferencias = require('./cruds/preferencias');
16 const registros = require('./cruds/registros')
17
18 app.use(cors());
19 app.use('/usuarios', usersRoutes);
20 app.use('/ambientes', ambientesRoutes);
21 app.use('/dispositivos', dispositivosRoutes);
22 app.use('/checkins', checkinRoutes);
23 app.use ('/roles', roleRoutes);
24 app.use('/rolesDoAmbiente', rolesDoAmbienteRoutes);
25 app.use('/preferencias', preferencias);
26 app.use('/registros',registos);
27
28 const PORT = process.env.PORT || 8080;
29
30 app.listen(PORT, () => {
31   console.log(`Server is running on port ${PORT}`);
32 });
33
34
35
```

Figura 61- Configuração da página do Backend da App

```

1  .App {
2      text-align: center;
3  }
4
5  .App-logo {
6      height: 40vmin;
7      pointer-events: none;
8  }
9
10 @media (prefers-reduced-motion: no-preference) {
11     .App-logo {
12         animation: App-logo-spin infinite 20s linear;
13     }
14 }
15
16 .App-header {
17     background-color: #282c34;
18     min-height: 100vh;
19     display: flex;
20     flex-direction: column;
21     align-items: center;
22     justify-content: center;
23     font-size: calc(10px + 2vmin);
24     color: white;
25 }
26
27 .App-link {
28     color: #61dafb;
29 }
30
31 @keyframes App-logo-spin {
32     from {
33         transform: rotate(0deg);
34     }
35     to {
36         transform: rotate(360deg);
37     }
38 }
39 /* Styles for the navigation bar */
40 .navbar {
41     background-color: #333;
42     padding: 10px;
43 }
44
45 .navbar ul {
46     list-style-type: none;
47     margin: 0;
48     padding: 0;
49     display: flex;
50     justify-content: flex-start;
51 }
52
53 .navbar li {
54     margin-right: 20px;
55 }
56
57 .navbar a {
58     color: white;
59     text-decoration: none;
60     padding: 8px 16px;
61     border-radius: 4px;
62     transition: background-color 0.3s;
63 }
64
65 .navbar a:hover {
66     background-color: #575757;
67 }
68

```

Figura 62- Configuração da página do AppCss

Apêndice G- Proposta Original do Projeto

Gestão de conflitos de preferências de utilizadores no ambiente

x	Inteligência artificial		Multimédia/Realidade aumentada/...
	Aplicações móveis		Redes e gestão de sistemas
	Plataformas web		*

Orientador: Pedro Filipe Fernandes Oliveira

Coorientador: Paulo Matos

Objetivo

Pretende-se o desenvolvimento e implementação de uma solução, que dadas as preferências dos diferentes utilizadores num determinado ambiente/espço (casa, trabalho, espaço público), efetue a gestão de conflitos das preferências de cada utilizador, nomeadamente quanto às preferências de conforto (temperatura, humidade, luminosidade, musical, etc.) a aplicar no espaço.

Detalhes

A mobilidade diária no quotidiano do ser humano, aliada à atratividade do aumento da automatização das tarefas quotidianas, vem também no contexto das casas inteligentes (Smart Homes) criar a necessidade da deteção e identificação das preferências do utilizador de forma automatizada para assim poder ser realizada a consequente adaptação das condições de conforto do ambiente/espço aos utilizadores presentes. Neste contexto e para ir de encontro à automatização de todo o processo, pretende-se o desenvolvimento de uma solução que permita de forma automática e sem qualquer interação do utilizador, a gestão de conflitos das diferentes preferências de cada utilizador que se encontre no espaço, para tal devem ser tidas em conta as diferentes hierarquias, assim como valores de segurança dos diferentes equipamentos atuadores (ar condicionado, radiadores, ventiladores, etc.) presentes no espaço. Para o desenvolvimento pretende-se tirar partido das diferentes valências da inteligência artificial, para tornar a solução dinâmica e preditiva.

Metodologia de trabalho

1. Estudo prévio de plataformas já existentes;
2. Levantamento de requisitos;
3. Desenho de protótipo/mockups da aplicação;
4. Seleção de Linguagem de programação e possíveis frameworks a utilizar;
5. Desenvolvimento da aplicação;
6. Implementação e testes, em contexto real;
7. Possíveis melhorias, após testes.